

CARTRIDGE & CLOUD · DOCUMENTACIÓN RC1

Cartridge & Cloud — UI Style Guide

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

ESTADO

**Normative direction / implementation partially
representative**

DOCUMENTO

19 / 19_UI_Style_Guide.md

AUTORÍA

VRM Games / Blas Luis Rocha González

VERSIÓN

1.0-consolidated-draft

FECHA DOCUMENTAL

2026-07-01

Control y metadatos del documento

Archivo fuente: 19_UI_Style_Guide.md
SHA-256 del Markdown: b34ca4db1eba7a536466b6a9a105d8d92396ade2108aa7dd99fa7d5d4eaebc1a
Tamaño del Markdown: 149,800 bytes
Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

title	Cartridge & Cloud — UI Style Guide
subtitle	Sistema visual, interacción, accesibilidad, localización e implementación de interfaz
author	VRM Games / Blas Luis Rocha González
date	2026-07-01
lang	es-ES
version	1.0-consolidated-draft
status	Normative direction / implementation partially representative
platform	PC / Steam
engine	Unity 6.3 LTS 6000.3.18f1 / URP 17.3.0
application_version	0.0.17

Índice

0. Propósito, autoridad y estado del documento
1. Alcance
2. Jerarquía documental
3. Reglas de interpretación y madurez
4. Genealogía histórica completa
5. Baseline v0.3: aportaciones vigentes
6. Baseline v0.4: continuidad y evidencia
7. Baseline v0.5: minimal UI y Sprint 15 pendiente
8. Baseline v0.6: UI runtime y defecto de input
9. Inventario de fuentes históricas y operativas
10. Inventario de documentos consolidados 00–18
11. Estado ejecutivo de la interfaz
12. Pantallas y capas implementadas
13. Paneles de gestión implementados
14. Mapas de input actuales
15. Ajustes de accesibilidad implementados
16. Tokens cromáticos históricos
17. Valores cromáticos implementados
18. Identidad visual de interfaz
19. Pilares de experiencia
20. Legibilidad
21. Claridad operativa
22. Jerarquía visual
23. Densidad progresiva
24. Consistencia
25. Feedback inmediato
26. Prevención y recuperación de errores
27. Separación entre UI y mundo
28. Prioridad de consumo de input
29. Ratón y puntero
30. Teclado
31. Mando futuro
32. Resoluciones objetivo
33. Escalado de Canvas y texto
34. Relaciones de aspecto y safe areas
35. Grid de layout y espaciado
36. Paleta corporativa consolidada
37. Paleta funcional
38. Contraste
39. Estados de control
40. Tipografía
41. Jerarquía tipográfica
42. Texto, casing y tono
43. Números, moneda, fechas y cantidades
44. Iconografía
45. Tooltips
46. Botones
47. Botones destructivos
48. Toggles
49. Sliders
50. Dropdowns
51. Tabs
52. Campos numéricos
53. Listas
54. Tablas
55. Tarjetas

- 56. Modales
- 57. Toasts y mensajes
- 58. Banners
- 59. Indicadores de carga
- 60. Estados vacíos
- 61. Estados bloqueados
- 62. Estados de error
- 63. MainMenu
- 64. Nueva partida y slots
- 65. Confirmaciones destructivas
- 66. Pausa
- 67. Ajustes
- 68. Opciones de audio
- 69. Opciones visuales
- 70. Accesibilidad
- 71. HUD principal
- 72. Dinero
- 73. Día y hora
- 74. Objetivos y procedimiento
- 75. Alertas
- 76. Operations
- 77. Inventario
- 78. Pedidos
- 79. Proveedores
- 80. Recepción
- 81. Displays
- 82. Productos
- 83. Reservas y carrito
- 84. Clientes
- 85. Cola
- 86. Checkout
- 87. Economía
- 88. Ledger
- 89. Cierre diario
- 90. Modo construcción
- 91. Preview válido e inválido
- 92. Selección y retirada de objetos
- 93. Tutorial y onboarding
- 94. Mensajes contextuales
- 95. Input exclusivity y click-through
- 96. Foco y pila de navegación
- 97. Navegación entre controles
- 98. Animaciones y transiciones
- 99. Relación con audio
- 100. Relación con VFX y feedback del mundo
- 101. Localización ES/EN
- 102. Expansión, wrapping y truncado
- 103. Accesibilidad visual y color
- 104. Accesibilidad motora y cognitiva
- 105. Alternativas a información sonora
- 106. Implementación actual en Unity
- 107. uGUI frente a UI Toolkit
- 108. Componentes compartidos y tokens
- 109. Convenciones de nombres
- 110. Rutas de assets
- 111. Catálogo de iconos y componentes
- 112. Requisitos UX consolidados
- 113. Criterios Vertical Slice relacionados con UI
- 114. Tests automatizados
- 115. Tests manuales y matriz de resoluciones
- 116. Evidencia y nomenclatura
- 117. Placeholders y deuda de UI
- 118. Criterios de Sprint 16
- 119. Criterios de Sprint 17

- | | |
|--|---|
| 120. Criterios de H6 | 152. Operations: Customer |
| 121. Playbook: revisión de una pantalla | 153. Operations: Settings |
| 122. Playbook: creación de un componente | 154. Feedback canvas de Sprint 16 |
| 123. Playbook: cerrar click-through | 155. Canvases técnicos serializados |
| 124. Playbook: integrar localización | 156. Ciclo de vida entre escenas |
| 125. Playbook: revisión de accesibilidad | 157. Refresco, rebuild y rendimiento |
| 126. Playbook: migrar UI técnica | 158. Persistencia de preferencias |
| 127. Playbook: añadir icono o fuente | 159. Taxonomía de claves de localización |
| 128. Playbook: aprobar un modal | 160. Interfaz de sistemas futuros |
| 129. Antipatrones y elementos prohibidos | 161. Matriz de resoluciones, escalas e idiomas |
| 130. Riesgos y decisiones de rollback | 162. Matriz de capas y bloqueo de input |
| 131. Glosario | 163. Definition of Ready por tipo de trabajo UI |
| 132. Historial de revisiones y mantenimiento | 164. Definition of Done por tipo de trabajo UI |
| 133. Estado de slot vacío | 165. Work packages recomendados |
| 134. Estado de slot válido | 166. Protocolo de revisión semanal de UI |
| 135. Slot recuperado desde backup | 167. Protocolo de regresión tras cambiar datos o dominio |
| 136. Slot con schema no soportado | 168. Protocolo de captura y referencia visual |
| 137. Slot corrupto sin backup | 169. Trazabilidad mínima de una pantalla |
| 138. Almacenamiento no disponible | 170. Perfil técnico: Sprint15RuntimeCompositionRoot |
| 139. Overlay sin sesión activa | 171. Perfil técnico: MainMenuSlotScreen |
| 140. HUD en BeforeOpen | 172. Perfil técnico: StoreHudScreen |
| 141. HUD durante Open | 173. Perfil técnico: Sprint15UiFactory |
| 142. HUD durante Closing y Closed | 174. Perfil técnico: StoreUiProjectionService |
| 143. Fila de panel de gestión | 175. Perfil técnico: UiNavigationState |
| 144. Panel Help | 176. Perfil técnico: AccessibilitySettingsService |
| 145. Burbuja de tutorial | 177. Perfil técnico: TutorialService |
| 146. Modal de confirmación | 178. Perfil técnico: Sprint15InputSystemBridge |
| 147. Operations: Guide | 179. Perfil técnico: Sprint15InputMapGate |
| 148. Operations: Shop | 180. Perfil técnico: Phase1OperationsScreen |
| 149. Operations: Delivery | 181. Perfil técnico: Phase1RuntimeUiFactory |
| 150. Operations: Stock | 182. Perfil técnico: Phase1FeedbackPresenter |
| 151. Operations: Displays | 183. Perfil técnico: escenas MainMenu, Store y StoreInitial |

- | | |
|---|--|
| 184. Inventario de assets UI observados | 203. Campaña QA: slots y recovery |
| 185. Copy de éxito | 204. Campaña QA: escalado extremo |
| 186. Copy de advertencia | 205. Campaña QA: localización |
| 187. Copy de error recuperable | 206. Campaña QA: accesibilidad sin audio/color |
| 188. Copy de error de datos | 207. Campaña QA: cambio de escena y overlays |
| 189. Copy destructivo | 208. Campaña QA: build externa |
| 190. Copy de bloqueo | 209. Campaña QA: contenido dinámico |
| 191. Copy de loading | 210. Campaña QA: acciones idempotentes |
| 192. Copy de vacío | 211. Severidad específica de defectos UI |
| 193. Copy de tutorial | 212. Checklist de aprobación de pantalla |
| 194. Copy de tooling interno | 213. Checklist de candidata H6 para UI |
| 195. Extracción de strings | 214. Matriz de prioridad, convivencia y descarte de mensajes |
| 196. Diseño de keys | 215. Versionado de tokens, componentes y contratos visuales |
| 197. Tablas fuente ES/EN | 216. Estrategia de migración desde UI generada en runtime |
| 198. Pseudolocalización | 217. Criterios para elegir uGUI, UI Toolkit o una solución híbrida |
| 199. Formatos culturales | 218. Conservación de las pantallas históricas de visión futura |
| 200. QA lingüística | 219. Plan de madurez de UI por fase |
| 201. Campaña QA: foco completo | 220. Paquete de evidencia de una candidata UI para H6 |
| 202. Campaña QA: propagación de puntero | 221. Retrospectiva y mantenimiento del sistema de interfaz |

0. Propósito, autoridad y estado del documento

Esta Guía de Estilo de Interfaz define el lenguaje visual, interactivo, técnico y de accesibilidad de **Cartridge & Cloud**. Su función no es sustituir al `05_UX_Flow.md`, al GDD, al TDD, al QA Plan ni a la Art/Audio Bible. Convierte sus decisiones en reglas de interfaz verificables: cómo se presenta la información, cómo se consume el input, cómo se estructura un panel, qué estados debe mostrar un control, qué evidencia permite aprobar una pantalla y qué diferencias existen entre una referencia histórica, un prototipo runtime y una interfaz candidata a H6.

El documento es normativo para nuevos trabajos de UI y para la revisión de lo ya implementado. Cuando una regla visual entre en conflicto con un requisito funcional, prevalece el requisito funcional; cuando una comodidad de implementación entre en conflicto con accesibilidad, input exclusivo o integridad de datos, prevalece la seguridad de la interacción. El estado del documento es **consolidado, amplio y vigente como dirección**, pero la implementación real conserva deuda: tipografía provisional, strings hardcodeados en inglés, ausencia de tablas de localización, UI generada por código, click-through abierto en Operations y coexistencia con canvases técnicos de escena.

1. Alcance

La guía cubre MainMenu, slots, HUD, Operations, inventario, pedidos, proveedores, recepción, displays, productos, reservas, clientes, cola, checkout, economía, jornada, construcción, tutorial, pausa, ajustes, accesibilidad, mensajes, confirmaciones y feedback. También establece una base para pantallas futuras —empleados, investigación, ecommerce, publishing, desarrollo interno, plataforma e infraestructura— sin convertirlas en compromiso del vertical slice.

Quedan fuera de aprobación actual el diseño final para mando, Steam Deck, touch, VR/XR, UI de lanzamiento, marketing web, overlay de Steam y herramientas internas de editor. Pueden documentarse como restricciones de evolución. La guía sí exige que la arquitectura actual no impida localización, navegación por teclado, escalado, reducción de movimiento, separación UI/mundo y sustitución gradual de la UI técnica.

2. Jerarquía documental

La interfaz se gobierna mediante una cadena explícita. `00_Enfoque_y_Alcance.md` decide qué producto se está construyendo; `01_Game_Design_Document.md` define fantasía y sistemas; `02_Vertical_Slice_Specification.md` fija aceptación; `03_Technical_Design_Document.md` y `04_Modelo_de_Datos.md` fijan contratos; `05_UX_Flow.md` define recorridos y prioridades de

input; [07_QA_Testing_Plan.md](#) y [08_QA_Testing_Matrix.xlsx](#) definen validación; [17_Art_Bible.md](#) fija identidad visual; [18_Audio_Bible.md](#) fija feedback sonoro; esta guía define presentación e interacción.

Los Excel [12](#) y [13](#) registran estado y trazabilidad, no redefinen la experiencia. Las versiones históricas se conservan como genealogía. El código, las escenas y los assets son evidencia de implementación, no autoridad automática: una conducta existente puede ser deuda o placeholder.

3. Reglas de interpretación y madurez

Cada decisión se clasifica como **Vigente**, **Implementada**, **Implementada parcialmente**, **Placeholder**, **Diferida**, **Sustituida**, **Histórica** o **Visión**. “Implementada” significa que existe en el proyecto y cuenta con evidencia; no implica acabado visual. “Vigente” significa que debe guiar el trabajo aunque todavía no exista. “Placeholder” permite continuar el flujo, pero no aprueba presentación. “Visión” protege una posibilidad futura sin autorizar producción actual.

Una captura conceptual, un mockup o una tabla histórica no demuestra integración. Un test automatizado verde no demuestra legibilidad, ausencia de clipping, navegación de foco ni comportamiento a distintas resoluciones. Una UI visualmente atractiva no se aprueba si propaga input al mundo, representa mal el estado, permite una operación destructiva accidental o depende solo del color.

4. Genealogía histórica completa

La genealogía UI empieza con [Cartridge_And_Cloud_UI_Style_Guide_v0.3.md](#) en la baseline v0.3 y su reedición en v0.4. Esa versión estableció la identidad negra/verde, tokens detallados, escala 1920×1080, componentes, estados, BuildMode y un mapa muy amplio de pantallas futuras. También incluyó grandes bloques de publishing, desarrollo interno, ecommerce, plataforma, infraestructura y mercado; esos bloques son visión de producto y no alcance actual.

La v0.4 de la guía, publicada con la baseline v0.5, redujo la dirección a principios ejecutables y reconoció que MainMenu y ReturnButton eran técnicos. La v0.5, en baseline v0.6, documentó la UI runtime de Sprint 15, la jerarquía HUD/Operations/modales y el defecto de click-through. La presente consolidación no borra ninguna etapa: conserva las ideas útiles, identifica lo sustituido y vincula la dirección con el proyecto real.

5. Baseline v0.3: aportaciones vigentes

De v0.3 permanecen vigentes la identidad tecnológica oscura con acento verde, la necesidad de una capa comercial cálida, el principio de una sola acción primaria por panel, la escala de espaciado 4/8/12/16/24/32, los estados completos de control, la alineación de cifras, las unidades visibles, el feedback de BuildMode y la regla de que el color nunca es el único indicador.

No se adopta literalmente toda su densidad. Los paneles futuros descritos con tablas complejas, timelines, árboles e infraestructura deben rediseñarse cuando entren en alcance. La guía histórica es una biblioteca de patrones, no un backlog autorizado. Sus tokens se mantienen como origen del sistema cromático y se normalizan con los valores usados por la implementación actual.

6. Baseline v0.4: continuidad y evidencia

La reedición v0.4 mantuvo el contenido funcional de v0.3, pero añadió evidencia de la fundación Unity y distinguió especificación de implementación. Esa distinción se convierte aquí en una regla permanente: cada componente debe registrar estado visual, estado funcional y estado de QA por separado.

Una pantalla puede estar implementada y ser todavía técnica; un flujo puede estar aprobado conceptualmente y no existir; un control puede funcionar con ratón pero fallar con teclado. La revisión debe evitar expresiones ambiguas como “UI terminada” y utilizar resultados concretos: `Implemented` / `Visual placeholder` / `Keyboard pending` / `Build not validated`.

7. Baseline v0.5: minimal UI y Sprint 15 pendiente

La guía v0.4 de la baseline v0.5 documentó MainMenu y ReturnButton como UI técnica, estableció tokens simplificados y reservó Sprint 15 para la UI completa del vertical slice. Sus principios —legibilidad, jerarquía, paneles compactos, estados visibles, localización y no depender solo del color— siguen vigentes.

La implementación posterior confirmó la estrategia de runtime composition root. Sin embargo, la simplificación de cinco capítulos no reemplaza el detalle de v0.3. Esta guía vuelve a expandir componentes, accesibilidad, localización, QA y patrones sin reabrir sistemas futuros.

8. Baseline v0.6: UI runtime y defecto de input

La v0.5 histórica refleja la aplicación 0.0.17, Sprints 0–15 cerrados y Sprint 16 en curso. Define HUD reducido, Operations, modales destructivos, feedback contextual, contraste, scroll, tooltips y foco. Su hallazgo más importante es el botón **Vertical Slice Operations**: puede abrir la UI y producir simultáneamente movimiento al suelo.

La presente guía convierte ese defecto en requisito estructural. No basta con deshabilitar Gameplay después del callback del botón: el puntero debe reconocerse como consumido por UI **antes** de cualquier raycast, destino de movimiento, colocación o interacción de mundo. La solución debe cubrir orden de ejecución, EventSystem, action maps, pointer-over-UI y pruebas PlayMode.

9. Inventario de fuentes históricas y operativas

La tabla registra todas las fuentes extraídas específicamente para esta reconstrucción. Las copias repetidas se conservan porque permiten demostrar cuándo una decisión cambió de estado o fue reeditada sin modificación funcional.

Fuente	Categoría	Bytes	SHA-256
Documentation__00_Official_Baseline__v0.3__99_Source_Markdown__Cartridge_And_Cloud_Initial_Content_Catalog_v0.1.md	Catálogo histórico	23013	5b416cba7975a0ee42762eabb9ea236c30a19091fb3d3031285ac67ab0a83dcb
Documentation__00_Official_Baseline__v0.3__99_Source_Markdown__Cartridge_And_Cloud_Localization_Plan_v0.3.md	Localization histórico	3806	f48dcd0d33e717b3e65734a9f87677d3b72edb9e67b54ff630720ecc46686349
Documentation__00_Official_Baseline__v0.3__99_Source_Markdown__Cartridge_And_Cloud_Package_Manifest_v0.1.md	Manifiesto	6551	58b7f326720277d2b490784a8091cff34f3182ac4fff57796e70d7817d9a4bc1
Documentation__00_Official_Baseline__v0.3__99_Source_Markdown__Cartridge_And_Cloud_UI_Style_Guide_v0.3.md	UI Style Guide histórica	47855	d446fa1988af7343cacee8c3817afd3e3fd10f933fe1524d7e0b65f718842065
Documentation__00_Official_Baseline__v0.3__99_Source_Markdown__Cartridge_And_Cloud_UX_Flow_v0.3.md	UX Flow histórico	87463	abb29bbf9b9e9dfbef5161feb03ec07c7bc405cfe4b217e8843534279eb3b50c
Documentation__00_Official_Baseline__v0.3__PACKAGE_MANIFEST.md	Manifiesto	6098	7fb1bdbd0b11a372ccf4ad87efea6f88326188a21a8f4760680b0e49cd928783
Documentation__00_Official_Baseline__v0.4__99_Source_Markdown__Cartridge_And_Cloud_Initial_Content_Catalog_v0.1.md	Catálogo histórico	23118	2bb04a31e3a2661e17e00f23ac0becc61b2c198a36fce87cf9f7aa1c76f07103
Documentation__00_Official_Baseline__v0.4__99_Source_Markdown__Cartridge_And_Cloud_Localization_Plan_v0.3.md	Localization histórico	3911	2158be2c46ec697e617419a9192bbaa8a0dd4435b5c650fca9295921b958cb3a

Fuente	Categoría	Bytes	SHA-256
Documentation__00_Official_Baseline__v0.4__99_Source_Markdown__Cartridge_And_Cloud_Package_Manifest_v0.2.md	Manifiesto	2456	ccf1ff8da3ecfb69e1ee93a902e1d13d706d5e7ada9ac4a185515c8f93cd2b7a
Documentation__00_Official_Baseline__v0.4__99_Source_Markdown__Cartridge_And_Cloud_UI_Style_Guide_v0.3.md	UI Style Guide histórica	47960	8e0d241b40ecc46860cc6c51036f436d85ecfe ca0cefc721897b52396b7fca2
Documentation__00_Official_Baseline__v0.4__99_Source_Markdown__Cartridge_And_Cloud_UX_Flow_v0.3.md	UX Flow histórico	87568	2f7e2355cb0e994394e536f8de514ba499aca16a2a81b8e6be1867a025842a9d
Documentation__00_Official_Baseline__v0.4__PACKAGE_MANIFEST.md	Manifiesto	2456	ccf1ff8da3ecfb69e1ee93a902e1d13d706d5e7ada9ac4a185515c8f93cd2b7a
Documentation__00_Official_Baseline__v0.5__06_Development_Records__Governance__Current_Project_Baseline_Record.md	Histórico/operativo	406	518e52784ae6c8737a360e221905ab542dd0a7c54d164409b71208556d26f793
Documentation__00_Official_Baseline__v0.5__06_Development_Records__Handoff__CURRENT_PROJECT_HANDOFF.md	Histórico/operativo	2275	9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
Documentation__00_Official_Baseline__v0.5__06_Development_Records__Sprints__Sprint_03__Governance__ADR-0012_Scene_Driven_Input_Contexts.md	ADR	1159	13c16537cc90fb0cdf3e74d588a585635c8b7b998433a1ed52501aad1d89af0f
Documentation__00_Official_Baseline__v0.5__06_Development_Records__Sprints__Sprint_03__Governance__ADR-0015_Project_Input_Actions_And_Context_Routing.md	ADR	1653	b665c0f33852bfb4ae7d0de42fac1636487181e19c28593c1d728d412e21954e
Documentation__00_Official_Baseline__v0.5__99_Source_Markdown__Cartridge_And_Cloud_Initial_Content_Catalog_v0.2.md	Catálogo histórico	2513	02439c64569896a24e292ec42717612412828e784bf7c1c701fbc477d57b0d51
Documentation__00_Official_Baseline__v0.5__99_Source_Markdown__Cartridge_And_Cloud_Localization_Plan_v0.4.md	Localization histórico	1892	01a3ddb286f975644985bc0c034eab75ef2ba4a283cd200f2e5dfd673732839f
Documentation__00_Official_Baseline__v0.5__99_Source_Markdown__Cartridge_And_Cloud_Package_Manifest_v0.3.md	Manifiesto	1783	7ae705ca39b1e39345748c208af4274cbbe9eaf887a249e5666bceefadf9d0d2
Documentation__00_Official_Baseline__v0.5__99_Source_Markdown__Cartridge_And_Cloud_UI_Style_Guide_v0.4.md	UI Style Guide histórica	2262	8b3346d21414eb370a5e98eb7871f5083cf7bc8d4214ab5cff7ae87ff0b46a22
Documentation__00_Official_Baseline__v0.5__99_Source_Markdown__Cartridge_And_Cloud_UX_Flow_v0.4.md	UX Flow histórico	3023	43143ab9e7504624ffdb4a2e9612332ee795f91ae05ea5d6448e5ce2f018bba8
Documentation__00_Official_Baseline__v0.5__CURRENT_PROJECT_HANDOFF.md	Histórico/operativo	2275	9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
Documentation__00_Official_Baseline__v0.5__PACKAGE_MANIFEST.md	Manifiesto	814	8bb540cf6f6eb6e6f079cf55786ab3b53f4ec042bb1dd01147edb03222f306c5
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Governance__Current_Project_Baseline_Record.md	Histórico/operativo	605	98e04d98696a645237fcf80cbca55467a16a09e6bd0d46df1a74637a222ca2e4
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Handoff__CURRENT_PROJECT_HANDOFF.md	Histórico/operativo	1241	7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87

Fuente	Categoría	Bytes	SHA-256
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Historical__Sprint_16_Integration_Timeline.md	Sprint 16	1783	89fde821c460328fbb8700022883006d2eb8a1ab892c266972b7a1cc207cbcc0
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Sprints__Sprint_16__Documentation__StoreInitial_Authoring_Plan.md	Sprint 16	741	f67d195bcdab2bd1f861dcf5c53d0a299260c1f04c2f299250856a76616131eba
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Sprints__Sprint_16__Governance__Sprint_16_Current_Status.md	Sprint 16	665	bc336c8bf852d942d1b0289cebffd9db889088990fcc6151df5db559e94d1a9
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Sprints__Sprint_16__Governance__Sprint_16_Representative_Asset_Integration_Record.md	Sprint 16	554	91133cf2823d732d58fcd23601d6816e610e00375cc38e39737929d88fc4ec54
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Sprints__Sprint_16__QA__S16_Acceptance_Matrix.md	Sprint 16	572	4ad529aceeaf7e2ea87def0e0be2f8f3998d6218f6ac11a06b40f770e66c44fd
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Sprints__Sprint_16__QA__S16_QA_Execution_Record.md	Sprint 16	328	40c6de3168db5dcf5c71afc166c0a3a2bf34f2dc3e71344c3ccea6176d573766
Documentation__00_Official_Baseline__v0.6__06_Development_Records__Sprints__Sprint_16__Traceability__S16_Traceability_Entries.csv	Sprint 16	460	3263ee5adec14e81588139e9c291ae973115ac75bef79233e6a514ac3643fd67
Documentation__00_Official_Baseline__v0.6__99_Source_Markdown__Cartridge_And_Cloud_Initial_Content_Catalog_v0.3.md	Catálogo histórico	2784	68f8841d7f5e83b1078bbf8c110e799165b74f5d75a823839d0acfd3086b333e
Documentation__00_Official_Baseline__v0.6__99_Source_Markdown__Cartridge_And_Cloud_Localization_Plan_v0.5.md	Localization histórico	1891	e8c63ec8a67f5d87f5dce078c94ac08620b27a064a5a6866a20fa78de1becf9
Documentation__00_Official_Baseline__v0.6__99_Source_Markdown__Cartridge_And_Cloud_Package_Manifest_v0.4.md	Manifiesto	1910	89ae5a7d733d8d70f36b64cfe3653aed99149827155fcddec0cadf650fd38acb
Documentation__00_Official_Baseline__v0.6__99_Source_Markdown__Cartridge_And_Cloud_UI_Style_Guide_v0.5.md	UI Style Guide histórica	2181	857f070207a752796ff5c21968020d3230a8c357d84a140593c74c47c59ee645
Documentation__00_Official_Baseline__v0.6__99_Source_Markdown__Cartridge_And_Cloud_UX_Flow_v0.5.md	UX Flow histórico	2707	5c755541bbc69510af12839f8435e090972edc7b7bb484139683614f69fcb85c
Documentation__00_Official_Baseline__v0.6__CURRENT_PROJECT_HANDOFF.md	Histórico/operativo	1241	7a297d84a36ecdd9e4296a631fc5acbfff882218d125e63312850e8f28b46cd87
Documentation__00_Official_Baseline__v0.6__PACKAGE_MANIFEST.md	Manifiesto	876	c25b2f638ce7814f8f2d9a153481efe6891a01b7e873f1a3251775ee51b922dd
Documentation__10_Development_Records__Handoff__CURRENT_PROJECT_HANDOFF.md	Histórico/operativo	1168	ea48666ee43dbf978bef364ee645cd3ff32f31e4e6dfbd96b9edfab316546028
Documentation__10_Development_Records__Sprint_03__Governance__ADR-0012_Scene_Driven_Input_Contexts.md	ADR	1159	13c16537cc90fb0cdf3e74d588a585635c8b7b998433a1ed52501aad1d89af0f
Documentation__10_Development_Records__Sprint_03__Governance__ADR-0015_Project_Input_Actions_And_Context_Routing.md	ADR	1653	b665c0f33852bfb4ae7d0de42fac1636487181e19c28593c1d728d412e21954e

Fuente	Categoría	Bytes	SHA-256
Documentation__10_Development_Records__Sprint_15__Architecture__ADR-0061- RuntimeUICompositionRoot.md	ADR	251	65c71a50143e1217e64a395d48e368db684e9051879a84fe991d293254644fd9
Documentation__10_Development_Records__Sprint_15__Architecture__ADR-0066- ExclusiveUIInput.md	ADR	193	dd52812f99015a563282f87506a4b57884e6c41b4f0f9488cfb4d58944d74bbe
Documentation__10_Development_Records__Sprint_16__Governance__Sprint_16_Two_Phase__ Charter.md	Sprint 16	640	b191b8b5d47654141df7860d6a79430552cc68e1bcd786d1d4fef55583dc605
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0067- Sprint16TwoPhaseDelivery.md	ADR	214	59f759e966b057f091473d52cf205d1a089a0917d2f8fff80342ca8ce2ca0a75
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0068- PurePhase1Sidecar.md	ADR	232	5a4b767d6daf552a48dc2b4d256b38c698da130b41314243902c4b5fbab68459
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0069- AutosaveCoordinatedCheckpoint.md	ADR	250	c4aaad9ce98b604d52894350668f7e59cc8c9d893ec3e96658a467bff0dbd44a
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0070- PlacementCompatibilityBridge.md	ADR	353	66a5b5172267c6984edd5474ba03dc2fb0da28de72c310ced74feba060f6df17
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0071- DecoupledPresentationFallbacks.md	ADR	223	f2e025be2f68b6de2388bd63a6b25549d4e6d111012bc685b034fa120ff9616c
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0072- SharedMaterialAndPrefabIds.md	ADR	223	f32e2b4b274522d03df3c1c08084d86114a19dbe15fa2b3008c217943e2ff11a
Documentation__10_Development_Records__Sprint_16__Phase_1__Architecture__ADR-0073- CompletedCheckoutSnapshotRecords.md	ADR	377	7d401e745e557c63167dae48cd0f4b05b1d08f5c58a2e652b4321925fc35caa7
Documentation__10_Development_Records__Sprint_16__Phase_1__Documentation__S16_P1_I mplementation_Record.md	Sprint 16	819	3538c025e899624c8dfb10c740bcb06387f5be41a78a3396a8f5937a5cfc147
Documentation__10_Development_Records__Sprint_16__Phase_1__Documentation__S16_P1_P reImplementation_Record.md	Sprint 16	496	0d6915434ed6282040553f42f849399d6a8e3380eb4e38a6082ea3d13fba0e9c
Documentation__10_Development_Records__Sprint_16__Phase_1__Governance__S16_P1_Chart er.md	Sprint 16	874	4b5190df961a1180ce9fed6ced5fe194e88998f5f65df176bea9104c7bc825dc
Documentation__10_Development_Records__Sprint_16__Phase_1__Governance__S16_P1_Curr ent_Status.md	Sprint 16	569	6494de60296d252da3368dab5a5850938c4a5c219d414f80bde328379f1c7f3b
Documentation__10_Development_Records__Sprint_16__Phase_1__QA__S16_P1_Acceptance_M atrix.md	Sprint 16	1130	d6d2ca42287be0fced9c3d9da09df5db9729442157035a9a711e0ddbb9d91337
Documentation__10_Development_Records__Sprint_16__Phase_1__QA__S16_P1_QA_Execution _Record.md	Sprint 16	443	a044ba4e7ebb4af622c74397e2a8f0ff6e59ad7e9c1bf89ddb62f5b06bcac7
Documentation__10_Development_Records__Sprint_16__Phase_1__QA__S16_P1_Test_Plan.md	Sprint 16	836	ff35438f414f45a152e04532664e348d0fed9bae380c9dc70bf520af5c7c04c8
Documentation__10_Development_Records__Sprint_16__Phase_1__QA__S16_P1_Validation_Ch ecklist.md	Sprint 16	646	ac85fc7fe9c1e905ca5ea2eb245aa016acf8c611b2a3fe1a10fb3d0740e0198b

Fuente	Categoría	Bytes	SHA-256
Documentation__10_Development_Records__Sprint_16_Phase_1_Traceability__S16_P1_Asset_Inventory.csv	Sprint 16	6002	c91f4676f6c8b07c075fda4bf3563530380a5e35770d9f68b5497b77cf99641b
Documentation__10_Development_Records__Sprint_16_Phase_1_Traceability__S16_P1_Traceability.csv	Sprint 16	1635	d1e5ada71ed4d81b6a067aa05ea35626d5156967a1f8b212da829be32a4d318d

10. Inventario de documentos consolidados 00-18

Estos artefactos forman la capa documental vigente consultada por la guía. El hash identifica la copia auditada; deberá actualizarse cuando se publique una baseline consolidada posterior.

Documento	Bytes	SHA-256
00_Enfoque_y_Alcance.md	127401	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f
01_Game_Design_Document.md	132822	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177
02_Vertical_Slice_Specification.md	64531	75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
03_Technical_Design_Document.md	101994	66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
04_Modelo_de_Datos.md	102948	d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
05_UX_Flow.md	56805	e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e
06_Production_Roadmap_y_Sprint_Plan.md	61564	d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff
07_QA_Testing_Plan.md	79435	bc9c05a3737e345546ef16e5390e034295ef35ad66c31647852aec81b4e7affe
08_QA_Testing_Matrix.xlsx	185032	233e57f01d2468fe13c136c8fcd3ee75b39bd3de6530349626ea98d4ff254214
09_CSharp_Coding_Standards.md	95321	3b734d6cd49f31c09c10bb4941625e143f6c634e3fb50c157f0720e73b1c2a68
10_Unity_Project_Setup_Guide.md	78408	31430bb0544edd9577323aa0009d6ca3df8b9fbcf6c8a150fc33df69b9f963a4
11_Build_y_Versioning_Guide.md	85324	38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbe3f4cb1931d3e04f602215
12_Excel_Maestro_de_Produccion.xlsx	158100	405376cb64f49b34f1f842f3840c12654e9f08e2ec4534b5c149fa811e87dd83
13_Trazabilidad_y_Control_de_Cambios.xlsx	189424	f674e21d1a3a1f0970a3d339f26f2a2b51a19b5c257c24b9d3fa4118a5cf50eb
14_Project_Binder_Indice_Maestro.md	92828	0c24dcd47e0d44793e75e764e3fc2c146e5b1fea04d887d43feb0df558711559
15_Guia_Maestra.md	119920	879bc1925ccccf77e10df6c104302f3afc72f71e8c58dd85ae4f06e84e6f8b624
16_Auditoria_Global_de_Coherencia.md	87439	4a62376b8aee623be40957ae0511d2dcab2919947f3abb2cef3502b9bb60863e

Documento	Bytes	SHA-256
17_Art_Bible.md	133913	095c6cc42223a972c7faa68435595fc7c22f839fab62b2038371a0ef6c14239
18_Audio_Bible.md	144525	c2eabe93a7c26f70e1d4d77fb0ddcaa97e2cdd1e70247e637b9db5bd2f07ce92

11. Estado ejecutivo de la interfaz

La interfaz funcional de Sprint 15 existe y permite seleccionar slots, crear/cargar/borrar partidas, ver HUD, abrir paneles de gestión, recorrer tutorial, cambiar ajustes de accesibilidad, pausar, confirmar acciones y ejecutar transiciones de jornada. Sprint 16 añade un panel Operations separado para el procedimiento jugable y feedback audiovisual.

La presentación todavía es representativa: se construye con `UnityEngine.UI`, `Text` legacy, colores embebidos en C#, tipografía `LegacyRuntime.ttf`, strings ingleses hardcodeados y dos fábricas paralelas. No hay UXML, USS, TextMeshPro, design tokens como assets, atlas de iconos, fuente licenciada, localización integrada ni menú de audio definitivo. MainMenu, Store y StoreInitial conservan canvases técnicos serializados. El resultado es funcional, pero no una UI de lanzamiento.

12. Pantallas y capas implementadas

La siguiente matriz distingue las capas serializadas de las generadas en runtime. La coexistencia es deliberada durante la migración, pero debe revisarse para evitar duplicación visual, navegación simultánea o controles activos detrás de overlays.

Pantalla/capa	Tecnología	Fuente	Contenido	Estado
MainMenu scene legacy canvas	Scene-authored uGUI	MainMenu.unity	Title, subtitle, EnterStoreButton, QuitButton	Historical/technical; remains serialized
Sprint15MainMenuUI	Runtime-generated uGUI	MainMenuSlotScreen.cs	Three save slots, continue/new/delete, backup recovery, accessibility, help, quit	Implemented; overlay sorting order 5000
Store scene legacy canvas	Scene-authored uGUI	Store.unity / StoreInitial.unity	ReturnButton, scope notice, technical title	Historical/technical; coexists with runtime UI
Sprint15StoreUI	Runtime-generated uGUI	StoreHudScreen.cs	HUD, management panels, tutorial, pause, confirmation, autosave feedback	Implemented; overlay sorting order 5000
S16_P1_OperationsCanvas	Runtime-generated uGUI	Phase1OperationsScreen.cs	Guide, shop, deliveries, stock, displays, customer, settings	Implemented representative tool; sorting order 4550
S16_P1_FeedbackCanvas	Runtime-generated uGUI	Phase1FeedbackPresenter.cs	Transient audiovisual feedback messages	Implemented representative feedback

13. Paneles de gestión implementados

`ManagementPanelId` define diez paneles. La mayoría proyecta estado de dominio y no modifica datos; DayCycle y Accessibility exponen comandos. La futura UI debe mantener esa separación para evitar mutaciones implícitas al abrir o refrescar una vista.

Panel	Información	Interacción	Estado
Inventory	Stock totals, capacity and locations	Projection/read-only	Implemented
Suppliers	Orders, requested/received units and costs	Projection/read-only	Implemented
Displays	Assignments and available stock	Projection/read-only	Implemented
Customers	Active customers and patience	Projection/read-only	Implemented
Shopping	Sessions, carts and reservations	Projection/read-only	Implemented
Checkout	FIFO queue, station and completed transactions	Projection/read-only	Implemented
DayCycle	State, elapsed time and transition actions	Projection + commands	Implemented
Economy	Cash, revenue, supplier costs and gross result	Projection/read-only	Implemented
Help	Keyboard, panels and autosave guidance	Static text	Implemented
Accessibility	Scale, reduced motion, tutorial and destructive confirmations	Settings + commands	Implemented

14. Mapas de input actuales

El asset de Input System contiene mapas Player y UI. Los bindings amplios proceden de la plantilla de Unity; su existencia no equivale a soporte probado de touch, XR o gamepad.

Ma pa	Acciones	Bindings observados	Interpretación
Play er	Move, Look, Attack, Interact, Crouch, Jump, Previous, Next, Sprint	Keyboard/mouse, gamepad, touch, joystick/XR bindings inherited from template	Implemented; several bindings exceed current product scope
UI	Navigate, Submit, Cancel, Point, Click, RightClick, MiddleClick, ScrollWheel, tracked device position/orientation	Input System UI map	Implemented and enabled by bridge

15. Ajustes de accesibilidad implementados

Los valores pertenecen al modelo de dominio y se persisten mediante JSON. Deben considerarse parte del contrato de save/configuración de usuario, aunque la superficie de ajustes todavía sea técnica.

Ajuste	Rango	Default	Estado
UI scale	80-150%	100%	Persisted JSON; changes Canvas scale
Text scale	80-150%	100%	Persisted JSON; changes runtime font sizes
Reduced motion	On/Off	Off	Persisted; intent exists, visual transitions remain minimal
Message duration	1-30 seconds	6 seconds	Domain setting exists; not exposed in current settings panel
Tutorial enabled	On/Off	On	Persisted; tutorial progress is per slot
Destructive confirmations	On/Off	On	Persisted; can bypass confirmations

16. Tokens cromáticos históricos

La paleta histórica es la referencia semántica. No todos los valores aparecen literalmente en el runtime actual, pero sus roles siguen siendo válidos.

Token	HEX	Uso	Origen
VRM Black	#050505	Deep background	v0.3
Deep Charcoal	#0B0F0C	Bars and overlays	v0.3
Panel Graphite	#111812	Panels	v0.3
Control Surface	#16201E	Inputs and cards	v0.3
Border	#2F463D	Borders and separators	v0.3
VRM Green	#00C853	Primary action	v0.3
Neon Accent	#00FF66	Exceptional focus	v0.3
Soft Green	#7CFFB2	Soft positive	v0.3
Data Cyan	#49E6FF	Metrics/information	v0.3
Warning Amber	#F2C94C	Warning	v0.3

Token	HEX	Uso	Origen
Danger Red	#FF5C5C	Critical/error	v0.3
Text Primary	#F2F5F3	Primary text	v0.3
Text Muted	#A9BBB0	Secondary text	v0.3
VRM Green legacy	#008F46	Primary brand token	v0.4/v0.5
VRM Dark legacy	#101716	Dark background	v0.4/v0.5
VRM Gray legacy	#5B6660	Secondary text	v0.4/v0.5

17. Valores cromáticos implementados

Estos valores se derivan de colores serializados en las fábricas runtime. Deben migrarse a tokens centralizados; no se consideran una paleta definitiva separada.

Token observado	HEX aprox.	Uso	Fuente
Runtime background approx.	#040606	Main menu opaque background	MainMenuSlotScreen
Runtime content approx.	#060E0B	Main menu content panel	MainMenuSlotScreen
Runtime button normal approx.	#123024	Sprint 15 button normal	Sprint15UiFactory
Runtime button hover approx.	#14AD5E	Sprint 15 button highlighted/selected	Sprint15UiFactory
Runtime button pressed approx.	#056B33	Sprint 15 button pressed	Sprint15UiFactory
Phase 1 button normal approx.	#0A3D24	Operations button normal	Phase1RuntimeUiFactory
Title green approx.	#59FF9E	Runtime titles	MainMenu/Store/Operations

18. Identidad visual de interfaz

La UI combina precisión tecnológica con cercanía comercial. Los fondos oscuros y el verde VRM transmiten control; blancos cálidos, grafitos y acentos limitados evitan un aspecto de terminal agresiva. El mundo de la tienda puede ser más colorido que la interfaz, pero la UI debe permanecer reconocible en MainMenu, StoreInitial y futuras áreas empresariales. El verde identifica marca y acción; no debe inundar paneles ni competir con estados de éxito.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

19. Pilares de experiencia

Los pilares son claridad operativa, control reversible, feedback inmediato, densidad progresiva y continuidad con el mundo. La interfaz debe responder qué está pasando, qué puede hacerse, cuánto costará y qué consecuencia tendrá. Un jugador nuevo debe completar el Golden Path sin herramientas de debug; un jugador experto debe operar con pocos pasos y sin tutorial obligatorio.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

20. Legibilidad

La legibilidad se evalúa por contraste, tamaño, longitud de línea, jerarquía, espacio, movimiento y contexto. Texto visible sobre fondo del mundo necesita panel o sombra suficiente. Los valores críticos no pueden mezclarse con decoración. Ningún panel debe requerir memorizar qué significaba un color visto en otra pantalla.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

21. Claridad operativa

Cada acción debe usar un verbo concreto y mostrar objeto y consecuencia: `Order case`, `Delete slot`, `Complete Close`. Los estados no se redactan como mensajes genéricos. Un bloqueo debe explicar la condición incumplida y ofrecer una siguiente acción cuando exista.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

22. Jerarquía visual

La jerarquía usa posición, tamaño, peso, espaciado y contraste antes que color. Título identifica contexto; resumen muestra estado; contenido permite inspección; acciones se colocan al final o cerca del elemento afectado. Una acción primaria por bloque. Destructivas separadas espacialmente.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

23. Densidad progresiva

El HUD contiene solo estado de alta frecuencia. Operations y paneles contienen detalle. Tooltips y subpaneles ofrecen profundidad. No debe copiarse toda la información de dominio en una tabla única. El jugador puede pasar de señal a explicación y de explicación a acción.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

24. Consistencia

Mismos conceptos usan mismos nombres, posiciones y estados. `Close` cierra capa; `Cancel` abandona operación; `Back` vuelve al nivel anterior. Cash, stock, day y save deben formatearse igual en HUD, panel y resumen. Las dos fábricas runtime actuales deben converger para evitar divergencia.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

25. Feedback inmediato

Toda interacción produce confirmación perceptible en el mismo frame cuando sea posible: cambio de estado, pressed state, mensaje, audio o VFX. Las operaciones asíncronas muestran progreso. El feedback no debe afirmar éxito antes del commit de dominio.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

26. Prevención y recuperación de errores

Prevenir es preferible a corregir: deshabilitar acciones imposibles, mostrar límites y validar antes de confirmar. Cuando ocurre un error, conservar datos, explicar la causa y permitir reintento o cancelación. Los errores de save, schema, almacenamiento o escena requieren rutas de recuperación específicas.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

27. Separación entre UI y mundo

UI y mundo son contextos mutuamente excluyentes cuando una capa interactiva está abierta. El HUD pasivo no bloquea mundo; un panel, modal, tutorial interactivo o pausa sí. Un clic consumido por UI no puede convertirse en destino, placement, selección o interacción aunque la UI se cierre durante el mismo callback.

La aceptación de este principio requiere prueba manual en Editor y build, además de cobertura automatizada cuando la conducta pueda aislarse. Cualquier excepción debe registrarse como deuda con owner, gate y criterio de cierre.

28. Prioridad de consumo de input

El orden normativo es: sistema operativo/ventana → modal o confirmación → capa UI superior → panel UI → tutorial/tooltip interactivo → HUD interactivo → herramienta de mundo → navegación/cámara. `Escape/Cancel` actúa sobre la capa superior; no ejecuta simultáneamente pausa y cierre de modal. `Submit` activa el control enfocado una vez. Click, double click y press/release no deben disparar acciones duplicadas.

`Sprint15InputMapGate` cambia el contexto a UI y restaura el anterior. Esa protección es necesaria pero no suficiente para el primer evento que abre una capa. Debe añadirse una comprobación de EventSystem antes de los raycasts de mundo, o un consumo explícito en el router de input. La prueba debe simular el mismo frame de apertura.

29. Ratón y puntero

El ratón es el dispositivo principal de PC. Hover comunica interactividad; click primario confirma; rueda desplaza; click derecho puede cancelar en BuildMode solo si se documenta y no contradice menús contextuales. El target mínimo recomendado es 44×44 px efectivos a escala 100 %, con 54 px para acciones principales. Los controles no deben depender de precisión subpíxel.

El puntero sobre UI bloquea mundo incluso si el elemento visible no tiene callback propio pero pertenece a una superficie que captura el evento. Los textos no interactivos tienen `raycastTarget = false`; paneles que deban impedir click-through necesitan un `Graphic` raycasteable o un bloqueador equivalente.

30. Teclado

Tab y Shift+Tab recorren foco en orden lógico; Enter/Submit activa; Escape/Cancel cierra la capa superior; flechas pueden navegar listas, tabs o steppers. El foco inicial debe asignarse al control seguro más probable, nunca automáticamente a `Delete`, `Quit` o una confirmación destructiva.

La navegación automática de uGUI es aceptable como placeholder, pero debe auditarse en layouts dinámicos. Al cerrar un modal se restaura el foco al control que lo abrió. El foco visible necesita un indicador más allá del cambio de color de hover y debe conservarse a escalas 80–150 %.

31. Mando futuro

El mapa UI ya incluye bindings de gamepad, pero el mando no está aprobado. Las pantallas deben diseñarse sin hover obligatorio, con foco persistente, orden direccional estable y acciones equivalentes. No se deben publicar claims de soporte hasta completar prompts, remapeo, sensibilidad, scrolling, teclado virtual y campaña de QA.

El desarrollo actual no debe introducir controles imposibles de navegar con foco. La compatibilidad estructural es una restricción; la certificación se difiere.

32. Resoluciones objetivo

La referencia de diseño es 1920×1080. Deben probarse como mínimo 1280×720, 1600×900, 1920×1080, 2560×1440 y una relación ultrawide representativa. La configuración actual de Player conserva 1024×768 y ventana no redimensionable como deuda de setup; no debe reinterpretarse como resolución objetivo.

A 1280×720 ninguna acción crítica puede quedar fuera de pantalla. A 1440p la UI no debe resultar físicamente diminuta. El test incluye fullscreen/windowed, DPI del sistema cuando sea posible y rebuild de UI después de cambiar resolución.

33. Escalado de Canvas y texto

Las fábricas actuales usan `ScaleWithScreenSize`, referencia 1920×1080 y match 0.5. La preferencia UI scale modifica el `scaleFactor`; el text scale recalcula tamaños. Ambos ajustes deben combinarse sin clipping ni áreas interactivas desalineadas.

La guía mantiene 80–150 % como rango vigente. En 150 %, paneles deben scrollar o refluír; no reducir texto silenciosamente. En 80 %, los targets interactivos conservan tamaño usable. La escala se aplica a todas las capas, incluidas Operations, feedback, tutorial y modales; hoy las fábricas paralelas pueden no responder igual y requieren unificación.

34. Relaciones de aspecto y safe areas

El layout se ancla a bordes lógicos y usa márgenes seguros. En 16:9 se permiten barras laterales de gestión; en ultrawide el contenido no debe estirarse hasta líneas ilegibles. En 4:3 o 16:10 se prioriza verticalidad y scroll. La UI no debe cubrir puerta, checkout o puntos de interacción críticos sin permitir cierre rápido.

Aunque PC no tiene notch habitual, el concepto de safe area protege overlays, capturas, streaming y futuros dispositivos. Los márgenes base recomendados son 32–40 px a 1080p, escalados con Canvas.

35. Grid de layout y espaciado

El sistema usa múltiplos de 4 px: 4 para microseparación, 8 para separación interna, 12 para pares label/value, 16 para grupos, 24 para secciones y 32–40 para márgenes. Las implementaciones actuales usan paddings 10–36 y spacing 5–14; deben normalizarse gradualmente.

Las filas de datos mantienen altura suficiente para texto escalado y wrap. Los paneles principales usan encabezado, cuerpo scrollable y área de acciones. No se añaden `ContentSizeFitter` anidados sin prueba de estabilidad y rendimiento.

36. Paleta corporativa consolidada

La paleta consolidada conserva negros casi puros, grafitos verdes y `VRM Green` como acción. Se adopta `#00C853` como acento principal de marca y `#008F46` como verde profundo/legacy. `#00FF66` se reserva para foco excepcional o key art, no para texto largo. Los paneles oscuros deben diferenciarse por luminosidad y borde, no por verdes saturados.

Los valores implementados se consideran aproximaciones. El objetivo es trasladarlos a un asset o clase de tokens única, con nombres semánticos y sin colores literales dispersos.

37. Paleta funcional

Información usa cyan moderado, éxito usa verde suave, advertencia usa ámbar y error usa rojo. `Disabled` usa menor contraste pero mantiene legibilidad. Los colores funcionales se acompañan de icono, título o texto: `Warning · Low stock`, `Error · Save failed`, `Success · Order received`.

El verde de marca no debe significar simultáneamente selección, éxito, disponibilidad y precio bajo sin otra señal. Los datos financieros positivos y negativos necesitan signo, etiqueta y formato, no solo color.

38. Contraste

El contraste se mide sobre el fondo real, incluidos fondos semitransparentes sobre `StoreInitial`. El texto normal debe apuntar al menos a 4.5:1 y texto grande a 3:1; controles y foco necesitan separación perceptible. Estos umbrales son objetivo de accesibilidad y deben verificarse con herramientas, no por impresión.

Los overlays opacos actuales favorecen contraste, pero pueden ocultar demasiado mundo. Las futuras transparencias se prueban en zonas claras, oscuras y con VFX. Las capturas de QA incluyen el peor caso.

39. Estados de control

Cada control interactivo define Normal, Hover, Focused, Pressed, Selected, Disabled, Loading, Success, Warning y Error cuando sean aplicables. Hover y Focused no son equivalentes: el primero sigue puntero; el segundo sigue navegación. Pressed confirma respuesta inmediata. Disabled explica motivo mediante tooltip o texto cercano.

Los botones runtime ya implementan normal/highlighted/selected/pressed/disabled. Faltan tokens centralizados, indicador de foco inequívoco, loading y estados semánticos por componente.

40. Tipografía

La tipografía objetivo es una sans-serif altamente legible para cuerpo y controles, una display tecnológica moderada para títulos y una monoespaciada solo para IDs, logs o cifras tabulares. Toda fuente debe tener licencia, caracteres latinos completos, signos monetarios y cobertura ES/EN.

La implementación usa `LegacyRuntime.ttf`, un recurso built-in, como placeholder. Las carpetas `UI/Fonts` e `UI/Icons` solo contienen `.gitkeep`. No se considera tipografía final ni se compartirá una fuente sin licencia verificada.

41. Jerarquía tipográfica

A 1080p: título de pantalla 32–42 px; título de panel 24–32; subtítulo 20–24; cuerpo 16–20; metadata 14–16; microtexto solo si no es crítico y nunca por debajo de 12 px efectivo. La implementación actual usa 14–42 según pantalla y se escala con preferencia.

La jerarquía se mantiene por ratio, no por valores absolutos. El text scale 150 % debe preservar diferencias y permitir wrap. Títulos en mayúsculas se reservan para marca o encabezado corto.

42. Texto, casing y tono

El tono es directo, profesional y cercano. Botones usan verbos: `Continue`, `Order`, `Receive`, `Assign`, `Close`. Evitar mayúsculas sostenidas en párrafos. Los términos técnicos se explican mediante tooltip. Los errores no culpan al jugador.

La UI actual mezcla inglés y conceptos internos como `SPRINT 16 · PLAYABLE BLOCKOUT`; esos rótulos son tooling de desarrollo y no deben aparecer en una build pública. La localización futura definirá ES/EN sin concatenación de frases.

43. Números, moneda, fechas y cantidades

El dominio almacena dinero en unidades menores enteras. La UI formatea por moneda y locale, conserva signo y separadores, y evita floats. Stock y cantidades son enteros con unidad. Fechas de slot muestran zona o formato local; el texto actual `yyyy-MM-dd HH:mm UTC` es técnico.

No concatenar cantidades con fragmentos traducibles. Usar plantillas localizadas. Coste, ingreso y balance deben distinguirse visual y semánticamente. Valores desconocidos usan `—`, no `0`.

44. Iconografía

Los iconos reducen carga, pero nunca sustituyen texto crítico. Deben compartir grosor, perspectiva, esquinas y caja óptica. Estados usan pares icono+texto. Los seis PNG de productos son contenido de catálogo, no un sistema de iconos UI.

No existe atlas de iconos general. Antes de añadir iconografía masiva debe crearse catálogo, licencia, tamaños, variantes y fallback textual. No copiar iconos de marcas o plataformas reales.

45. Tooltips

Un tooltip explica término, regla, unidad o causa; no contiene una acción obligatoria. Aparece tras retraso razonable, permanece mientras hay hover/foco y se posiciona dentro de pantalla. Debe funcionar con teclado mediante foco.

Los paneles actuales carecen de sistema de tooltip dedicado. Las explicaciones se muestran como filas o ayuda estática. Implementar tooltips solo cuando exista gestión de foco, localización y pruebas de borde.

46. Botones

Primario para la acción principal; secundario para alternativas; terciario para baja prioridad; peligro para destrucción. Mantienen target mínimo, label verbal, estados completos y foco visible. No más de una acción primaria por bloque.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

47. Botones destructivos

Delete, Replace, Quit con pérdida y Return sin autosave requieren confirmación según ajustes. El modal nombra objeto, consecuencia y opción segura. Confirmar no debe ser el primer foco.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

48. Toggles

Representan estados binarios persistentes. Label describe la opción, control muestra On/Off además de color. El cambio debe poder revertirse y reflejar guardado.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

49. Sliders

Se reservan para valores continuos como volumen. Muestran valor numérico y permiten teclado. Para escalas discretas 80–150, los steppers actuales son aceptables; un slider no debe ocultar límites.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

50. Dropdowns

Se usan para opciones mutuamente excluyentes con más de cuatro valores, como resolución o idioma. Deben mostrar selección actual, soporte de teclado y lista contenida en safe area.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

51. Tabs

Cambian contenido dentro del mismo contexto. La tab activa se diferencia por forma, texto y foco. Operations implementa tabs con botones; necesita estado selected persistente y navegación por flechas.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

52. Campos numéricos

Precio, cantidad y volumen aceptan límites, unidad y validación. Evitar entrada libre sin formato. Los errores no descartan el valor anterior.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

53. Listas

Cada fila tiene identidad, estado y acción. Scroll conserva contexto; selección no cambia datos. Las listas vacías explican cómo generar contenido.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

54. Tablas

Cabecera fija, cifras alineadas a la derecha, unidades visibles, orden y filtros cuando sean necesarios. A 1280×720 deben permitir scroll sin ocultar la columna de identidad.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

55. Tarjetas

Agrupar un objeto como slot, producto o pedido. Título, metadata, estado y acciones se ordenan siempre igual. La tarjeta de slot implementada es el patrón de referencia funcional.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

56. Modales

Bloquean input del mundo y capas inferiores. Incluyen título, explicación, acción primaria/segura, acción de cancelación y foco restaurable. No se apilan modales arbitrariamente.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

57. Toasts y mensajes

Comunican eventos no bloqueantes. Duración respeta preferencia; mensajes críticos permanecen o requieren acción. No usar toast para pérdida de datos, schema incompatible o acción destructiva.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

58. Banners

Alertas persistentes como modo offline, build técnica o guardado recuperado se sitúan en una zona estable. No desplazan controles de forma inesperada.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

59. Indicadores de carga

Muestran operación y evitan doble submit. Si duración es indeterminada, usar indicador no porcentual; si es conocida, progreso real. Reducir movimiento respeta la preferencia.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

60. Estados vacíos

Explican qué falta, por qué importa y cómo crear el primer elemento. `No supplier orders` debe acompañarse de acceso al catálogo cuando sea posible.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

61. Estados bloqueados

Muestran requisito faltante, coste, dependencia o gate. No usar disabled silencioso. Una feature futura no aparece como control roto; se oculta o marca claramente como no disponible.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

62. Estados de error

Incluyen causa comprensible, datos afectados, posibilidad de reintento y referencia técnica opcional. El código o stack trace no se presenta como mensaje principal.

La implementación se revisará con ratón, teclado, escalas 80/100/150 %, 1280×720 y 1920×1080. Cuando el componente sea reusable, debe tener una única fuente de estilo y pruebas de estados.

63. MainMenu

La pantalla principal presenta marca, selección de slots y acciones globales. El overlay runtime actual es opaco y prioriza slots. Debe eliminar o desactivar interacciones del canvas técnico situado detrás. La acción predeterminada depende del estado: Continue en slot válido; New Game en vacío. Quit permanece separado.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

64. Nueva partida y slots

Existen tres slots independientes. Cada tarjeta distingue vacío, válido, recuperado, schema no soportado, corrupción sin backup y almacenamiento no disponible. Reemplazar o borrar requiere confirmación. La UI no debe crear dos veces ni perder tutorial/autosave markers de otro slot.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

65. Confirmaciones destructivas

El patrón actual cubre Replace, Delete, Quit, Return to MainMenu y Skip tutorial según preferencias. Debe conservar foco, bloquear mundo, indicar exactamente qué se pierde y no cerrar antes de conocer el resultado.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

66. Pausa

La pausa es una capa superior. Debe congelar o gobernar el tiempo según diseño, bloquear acciones de mundo, ofrecer Resume, ajustes, ayuda y Return. El texto actual afirma que la gestión se pausa; debe comprobarse que simulación y audio responden de forma coherente.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

67. Ajustes

Los ajustes se dividen en Audio, Visual, Controls, Accessibility y Language cuando entren en alcance. Aplicar, revertir y defaults deben ser claros. Los cambios seguros pueden aplicarse inmediatamente; resolución y modo de pantalla necesitan confirmación con timeout.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

68. Opciones de audio

Music, Ambience, Effects y UI usan controles separados, coherentes con Audio Bible. Operations ofrece pasos 0/50/100 como tooling representativo. La UI final requiere sliders, mute, valor, preview controlado y persistencia.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

69. Opciones visuales

Resolución, modo de pantalla, calidad y reducción de movimiento se presentan sin afirmar soporte no probado. El cambio no debe dejar ventana inaccesible. La escala UI se mantiene separada de resolución.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

70. Accesibilidad

UI scale, text scale, reduced motion, message duration, tutorial y destructive confirmations forman el mínimo. El panel actual no expone message duration; debe añadirse o justificarse. Los ajustes se pueden alcanzar desde MainMenu y pausa.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

71. HUD principal

Muestra Day, State, Cash, Customers, Queue y Save. Debe ser compacto, no tapar la tienda y actualizarse sin parpadeo. Save status no puede sugerir guardado cuando solo existe un checkpoint pendiente.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

72. Dinero

Cash se formatea por currency code. Los cambios importantes muestran delta y causa en contexto. El HUD muestra saldo; Economy muestra desglose. No animar contadores si Reduce Motion está activo.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

73. Día y hora

El HUD distingue día, estado y progreso temporal. Estados BeforeOpen/Open/Closing/Closed/Results no dependen de color. Las acciones de transición viven en DayCycle y explican bloqueos.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

74. Objetivos y procedimiento

Sprint 16 Operations incluye una guía de pasos. En una UI final, objetivos deben ser jugables, no mencionar sprints o QA. La guía muestra paso actual, instrucción y condición de avance.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

75. Alertas

Stock bajo, cola, pedidos, save y cierre usan prioridad. Una alerta crítica no compite con toasts menores. Deben agregarse y deduplicarse para evitar spam.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

76. Operations

Operations es una herramienta representativa con tabs Guide, Shop, Delivery, Stock, Displays, Customer y Settings. Se abre en el lateral derecho y bloquea Gameplay. El defecto de click-through debe cerrarse antes de aprobar Sprint 16.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

77. Inventario

El panel muestra on hand, capacidad y ubicaciones. Las transferencias futuras deben ser explícitas, atómicas y mostrar origen/destino. El estado vacío ofrece acceso a pedidos o recepción.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

78. Pedidos

El catálogo presenta producto/mueble, cantidad, coste unitario/por caja y coste total. La acción Order debe deshabilitarse con motivo cuando falta dinero o proveedor. La confirmación se basa en preflight.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

79. Proveedores

La lista distingue proveedor, pedido, estado, unidades solicitadas/recibidas y costes. Los estados históricos no se traducen con concatenación. Futuros filtros no entran en H6.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

80. Recepción

Delivery muestra entregas pendientes y acción Receive. El resultado debe confirmar unidades y destino. Un fallo conserva el pedido y explica capacidad o estado incompatible.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

81. Displays

Displays muestra asignación, capacidad y stock. Asignar/reasignar no mueve inventario por sí mismo salvo que el contrato lo defina. La UI usa producto, fixture y estado inequívocos.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

82. Productos

Nombre, categoría, precio, stock e icono se presentan con marcas ficticias. Los seis iconos actuales pueden usarse como contenido, con fallback textual. No dependen de marcas reales.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

83. Reservas y carrito

Shopping muestra sesión, carrito y reserva temporal. La expiración necesita feedback temporal comprensible. La UI nunca promete una reserva si el commit falló.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

84. Clientes

El panel muestra estado, intención y paciencia sin convertir clientes en hojas de cálculo intrusivas. La información necesaria para debugging puede permanecer en tooling, pero la UI jugable usa señales resumidas y mundo.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

85. Cola

Queue muestra longitud y orden. El mundo debe comunicar formación de cola; el HUD solo alerta cuando requiere atención. No se reordena visualmente sin que el dominio lo confirme.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

86. Checkout

Muestra estación, cola, transacción y resultado. Preflight, commit e idempotencia se reflejan mediante estados; un doble click no vende dos veces. Venta completada se muestra después de persistir el registro correspondiente.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

87. Economía

Economy separa cash, revenue, supplier/received costs y gross result. El signo, unidad y periodo son visibles. Las cifras no usan color como única señal.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

88. Ledger

El ledger futuro registra entrada, tipo, importe, referencia y balance. Se diseña para auditoría, no para manipulación directa. Filtros y exportación son post-H6.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

89. Cierre diario

El flujo BeforeOpen→Open→Closing→Closed→Results muestra condiciones. `Complete Close` no se habilita si hay operaciones pendientes. Results resume ingresos, costes, clientes y autosave.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

90. Modo construcción

Entrar en BuildMode cambia contexto, muestra catálogo, coste, rotación y controles. Salir restaura gameplay. Un modal abierto bloquea placement.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

91. Preview válido e inválido

Verde válido, rojo inválido y ámbar advertencia se acompañan de icono/texto y motivo. Huella, rotación y posición final coinciden. Colorblind mode no depende del matiz.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

92. Selección y retirada de objetos

Objeto seleccionado muestra identidad y acciones permitidas. Retirar informa reembolso o pérdida. No se permite retirar arquitectura fija mediante UI dinámica.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

93. Tutorial y onboarding

El tutorial actual tiene pasos Welcome, Movement, Management, Inventory, Suppliers, Displays, Customers/Shopping, Checkout, DayCycle, Economy y Autosave. Es reinicializable, saltable y persistente por slot. Los textos deben localizarse y los anchors validar existencia.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

94. Mensajes contextuales

El mensaje aparece cerca del contexto sin ocultar la acción. Debe diferenciar éxito, información, advertencia y error; respetar duración; evitar spam; y conservar historial mínimo si es crítico.

Criterio transversal: la pantalla debe poder cerrarse mediante Cancel, mantener foco lógico, no propagar input al mundo, soportar texto expandido y mostrar estado real del dominio.

95. Input exclusivity y click-through

La resolución del defecto abierto requiere una defensa en profundidad. Primero, los action maps cambian a UI cuando se abre una capa. Segundo, los drivers de mundo consultan contexto y `EventSystem.current.IsPointerOverGameObject()` antes de procesar click. Tercero, las superficies UI relevantes son raycasteables. Cuarto, el evento que abre la capa se marca consumido o se difiere el cambio de modo para impedir que el mismo press llegue a mundo. Quinto, PlayMode cubre pointer down/up, click rápido, doble click y cierre en el mismo frame.

El criterio de cierre no es “ya no lo reproduce el autor”. Debe pasar en Store y StoreInitial, con Operations, HUD, modal, pausa, tutorial y paneles, en Editor y build.

96. Foco y pila de navegación

`UiNavigationState` modela capas Hud, ManagementPanel, Submenu, Tooltip, Confirmation y PauseMenu. La UI runtime todavía gestiona varias referencias directamente. La evolución debe adoptar una pila única con foco de retorno. Cada Push almacena el target que abrió la capa; Pop restaura ese target si sigue válido.

No se permite que dos modales tengan foco simultáneo ni que el mundo reciba Submit. Al cambiar de escena se limpia la pila y se destruyen overlays anteriores. Un control destruido requiere fallback seguro.

97. Navegación entre controles

El orden sigue lectura: título → contenido → acciones. En tabs, izquierda/derecha cambia tab y Tab entra al contenido. En listas, flechas mueven selección; PageUp/PageDown son opcionales. Los steppers usan botones —/+ y teclas direccionales. Scroll con teclado mantiene foco visible.

`Navigation.ModeAutomatic` es provisional. QA debe detectar saltos a controles invisibles, atrás en orden, ciclos o foco perdido después de rebuild dinámico.

98. Animaciones y transiciones

Las transiciones comunican relación espacial y cambio de estado; no decoran por defecto. Paneles pueden aparecer en 120–200 ms, modales con fade/scale sutil y toasts con entrada breve. No retrasar respuesta funcional. `Reduce Motion` elimina desplazamiento, parallax y contadores animados, conservando cambios instantáneos o fades mínimos.

La UI actual casi no anima; eso es aceptable para el vertical slice. Añadir animación no es requisito de Sprint 16 si introduce riesgo de input, clipping o rendimiento.

99. Relación con audio

UIConfirm y UiError son clips actuales. La UI final debe usar eventos semánticos, no reproducir sonido directamente desde cada botón. Submit, cancel, error y éxito se distinguen sin saturación. Hover no necesita audio por defecto. Los controles de volumen respetan canales Music, Ambience, Effects y UI.

Todo evento crítico tiene alternativa visual. Reducir volumen o silenciar UI no elimina información. La Audio Bible gobierna mezcla, concurrencia y licencias.

100. Relación con VFX y feedback del mundo

La UI no duplica innecesariamente el VFX del mundo. Placement válido puede usar ghost, color, texto y audio; checkout puede usar animación de estación y feedback HUD. Los VFX no deben quedar detrás de un overlay opaco si son necesarios para entender resultado.

`Phase1FeedbackPresenter` crea un canvas propio y mensajes de color. Debe integrarse con la jerarquía de toasts y preferencias de duración para evitar una tercera capa de estilo independiente.

101. Localización ES/EN

Español es idioma de desarrollo y QA; inglés es objetivo comercial mínimo. Todo texto final usa keys, parámetros y formatos por locale. IDs, enums y save no se traducen. No concatenar fragmentos como `Day` + número + `.` + estado. Las frases actuales hardcodeadas son placeholder.

El paquete Localization [1.5.12](#) está instalado, pero [Content/Localization](#) no demuestra tablas integradas. La guía exige crear tablas, selector de idioma, fallback, pseudo-localización y QA de build antes de H6 si ES/EN forma parte del gate; de lo contrario, registrar explícitamente alcance y deuda.

102. Expansión, wrapping y truncado

Los layouts se diseñan para +30 % de longitud y prueban pseudo-localización con diacríticos. Botones permiten wrap o anchura flexible; tabs cortas pueden usar abreviación aprobada. Los valores críticos nunca se truncan sin tooltip o acceso completo. Ellipsis se reserva para nombres secundarios.

[Text](#) runtime usa horizontal wrap y vertical overflow, lo que puede desbordar filas. Cada componente necesita altura dinámica o límites. A 150 % text scale y 1280×720 se revisan todas las pantallas.

103. Accesibilidad visual y color

La UI ofrece escala, texto y señales redundantes. Estados se diferencian por icono, palabra y forma. Las simulaciones de deuteranopia, protanopia y tritanopia revisan BuildMode, alertas y gráficos. No se promete un modo daltonismo completo hasta disponer de presets o patrones validados.

El título verde sobre fondo oscuro tiene buen potencial, pero la aprobación requiere medición. Los grises disabled no pueden ocultar labels esenciales.

104. Accesibilidad motora y cognitiva

Targets amplios, navegación de teclado, confirmaciones opcionales, tiempos ajustables y tutorial reiniciable reducen barreras. No exigir doble click, hold preciso o acciones simultáneas. Mensajes usan frases cortas, orden lógico y una acción recomendada.

La velocidad temporal y pausa se gobiernan desde diseño; una persona debe poder leer un panel sin perder progreso. Tooltips no desaparecen demasiado rápido. Los errores explican recuperación.

105. Alternativas a información sonora

Puerta, pedido, checkout, cierre y save no dependen solo de audio. HUD, toast, animación o icono muestran el mismo evento. Los subtítulos de diálogos no son necesarios si no hay voz, pero los indicadores sonoros relevantes deben tener equivalencia visual.

La UI final puede incluir intensidad de feedback y duración de mensajes. El audio silenciado sigue siendo una configuración plenamente operable.

106. Implementación actual en Unity

La UI usa uGUI (`UnityEngine.UI`) y se genera principalmente por C#. `Sprint15RuntimeCompositionRoot` detecta `MainMenu` y `Store`, destruye pantallas runtime previas y crea `MainMenuSlotScreen` o `StoreHudScreen`. Sprint 16 instala `Operations` y `Feedback` mediante un root separado. Las escenas ya contienen `EventSystem` con `InputSystemUIInputModule` y canvases técnicos.

Las fábricas crean `Canvas` `Screen Space Overlay`, `CanvasScaler`, `GraphicRaycaster`, `Images`, `legacy Text`, `Buttons`, `LayoutGroups`, `ScrollRects` y `Masks`. Este enfoque permitió cerrar Sprint 15 sin reautoría de escenas, pero dispersa tokens, dificulta edición visual, localización y reutilización. La transición puede ser hacia prefabs uGUI compartidos o UI Toolkit, con decisión ADR y migración incremental.

107. uGUI frente a UI Toolkit

No se impone una migración inmediata. uGUI es estable para overlays y input actual; UI Toolkit puede mejorar estilos, templates y authoring, pero introduce coste, compatibilidad y pruebas. La decisión se toma por necesidades reales: complejidad de tablas, tooling, accesibilidad, rendimiento, workflow y soporte runtime.

Para H6, corregir deuda crítica en uGUI es preferible a reescribir. Después de H6 se puede crear un ADR comparando: prefabs + `ScriptableObject` tokens + `TextMeshPro` frente a UI Toolkit UXML/USS. Hoy no existen UXML ni USS en Assets.

108. Componentes compartidos y tokens

Debe existir una fuente única para colores, tipografía, spacing, tamaños, audio UI, duraciones y estilos de control. Como mínimo: `UiThemeAsset`, `UiTypographyAsset`, `UiMotionSettings`, `UiIconCatalog` y prefabs/componentes compartidos o equivalentes.

Las dos fábricas actuales (`Sprint15UiFactory` y `Phase1RuntimeUiFactory`) duplican botón, texto, panel, layout y canvas con valores distintos. La consolidación es deuda A2/Sprint 17 salvo que afecte click-through o accesibilidad, en cuyo caso sube de prioridad.

109. Convenciones de nombres

Assets usan prefijo `CC_UI_` para temas y prefabs, `ui.` para localization keys y nombres semánticos para GameObjects. Evitar `Panel1`, `Text`, `Button` como identidad única. Los eventos usan verbo/objeto. Los IDs de control no cambian sin migrar tests o navegación.

Ejemplos: `CC_UI_Button_Primary.prefab`, `CC_UI_Theme_Default.asset`, `ui.main_menu.slot.continue`, `ui.store.checkout.complete`. El nombre visible se localiza; el ID técnico permanece estable.

110. Rutas de assets

La ruta objetivo es `Assets/_Project/UI/` con subcarpetas `Themes`, `Fonts`, `Icons`, `Prefabs`, `Screens`, `Localization` y `Tests` si se adopta. Los product icons permanecen en `Art/Textures/Products/Icons`. Audio UI permanece en `Audio/UI`.

No crear una carpeta `Resources` general para UI salvo requisito justificado. Las referencias deben estar en registry, prefab o addressable cuando se adopte. Las licencias viven en documentación legal y catálogo.

111. Catálogo de iconos y componentes

El catálogo registra ID, nombre, rol, estado, asset, licencia, tamaño, variantes, pantalla y fallback. Un componente registra props, estados, tokens, navegación, accesibilidad, localización y tests. El catálogo no es solo inventario visual; permite detectar duplicados y componentes sin owner.

Los seis iconos de producto se incorporan como `product.*.icon`, no como iconos genéricos. Las carpetas `UI/Fonts` e `UI/Icons` vacías se consideran deuda explícita, no evidencia de contenido.

112. Requisitos UX consolidados

`05_UX_Flow.md` contiene 71 IDs UX. Esta guía los agrupa como contrato de navegación, slots, input, construcción, inventario, pedidos, jornada, clientes, checkout, economía, save, accesibilidad, localización y Golden Path.

#	Requirement ID
1	UX-ACC-001
2	UX-ACC-002
3	UX-ACC-003
4	UX-ACC-004
5	UX-ACC-005
6	UX-BLD-001
7	UX-BLD-002
8	UX-BLD-003
9	UX-BLD-004
10	UX-BLD-005
11	UX-BLD-006
12	UX-BLD-007
13	UX-BLD-008
14	UX-BLD-009
15	UX-BOOT-001
16	UX-BOOT-002
17	UX-BOOT-003
18	UX-BOOT-004
19	UX-BOOT-005
20	UX-BOOT-006
21	UX-CHK-001
22	UX-CHK-002

#	Requirement ID
23	UX-CHK-003
24	UX-CHK-004
25	UX-CHK-005
26	UX-CUS-001
27	UX-CUS-002
28	UX-CUS-003
29	UX-CUS-004
30	UX-CUS-005
31	UX-DAY-001
32	UX-DAY-002
33	UX-DAY-003
34	UX-DAY-004
35	UX-DSP-001
36	UX-DSP-002
37	UX-DSP-003
38	UX-ECO-001
39	UX-ECO-002
40	UX-ECO-003
41	UX-GP-001
42	UX-GP-002
43	UX-GP-003
44	UX-GP-004
45	UX-GP-005
46	UX-GP-006
47	UX-INP-001
48	UX-INP-002
49	UX-INP-003

#	Requirement ID
50	UX-INP-004
51	UX-INP-005
52	UX-INP-006
53	UX-INV-001
54	UX-INV-002
55	UX-INV-003
56	UX-INV-004
57	UX-LOC-001
58	UX-ORD-001
59	UX-ORD-002
60	UX-ORD-003
61	UX-SAV-001
62	UX-SAV-002
63	UX-SAV-003
64	UX-SAV-004
65	UX-SLOT-001
66	UX-SLOT-002
67	UX-SLOT-003
68	UX-SLOT-004
69	UX-SLOT-005
70	UX-SLOT-006
71	UX-SLOT-007

La presencia de un ID en esta tabla no significa que su evidencia esté completa. Debe enlazarse a caso, ejecución, build y evidencia según [13_Trazabilidad_y_Control_de_Cambios.xlsx](#).

113. Criterios Vertical Slice relacionados con UI

Los siguientes criterios son especialmente vinculantes para la guía. Los requisitos APP se incluyen porque MainMenu y slots son superficies UI críticas.

#	Requirement ID
1	VS-APP-001
2	VS-APP-002
3	VS-APP-003
4	VS-APP-004
5	VS-INP-001
6	VS-INP-002
7	VS-UI-001
8	VS-UI-002
9	VS-UI-003
10	VS-UI-004

114. Tests automatizados

La base contiene tests EditMode para accesibilidad, autosave, slots, proyección, tutorial, navegación, authoring y repositorios; PlayMode para runtime de Sprint 15; e integración de Input System. Estos tests protegen invariantes y wiring, pero no sustituyen inspección visual.

La suite debe añadir click-through en el frame de apertura, restauración de foco, modal top-layer, rebuild tras escala, strings localizados, ausencia de doble submit, screen change con overlay y compatibilidad StoreInitial. Los tests no deben depender de textos ingleses cuando se integre localización; usar IDs o componentes.

115. Tests manuales y matriz de resoluciones

La campaña mínima recorre MainMenu, tres slots, crear/cargar/borrar, backup recovery, ajustes, Store, HUD, todos los paneles, tutorial, Operations, modal, pausa, Return y Quit. Se repite a 1280×720 y 1920×1080; escalas 80/100/150; ratón y teclado; ES/EN cuando estén disponibles.

Se registran clipping, foco, scroll, contraste, input, estado, texto, audio, feedback, performance y Player.log. La build externa es obligatoria para H6. Un resultado visual **PASS** incluye capturas y versión identificada.

116. Evidencia y nomenclatura

Cada evidencia incluye Requirement/Test ID, build, commit, resolución, escala, idioma, dispositivo, fecha y resultado. Capturas usan nombres como **UI-VS-INP-001_BLD-H6_1920x1080_100_ES_PASS.png**. Videos cortos demuestran foco, click-through o transición.

No recortar una captura de forma que oculte resolución o contexto. Player.log y test results se archivan junto a la build. Los mockups se etiquetan **CONCEPT**, no **PASS**.

117. Placeholders y deuda de UI

Son placeholders actuales: LegacyRuntime.ttf, strings hardcodeados en inglés, labels de Sprint 16, colores embebidos, dos fábricas duplicadas, paneles técnicos de escena, steps de volumen 0/50/100, product icons sin catálogo UI, ausencia de localización y falta de tooltip/icon system. Deben registrarse con severidad y gate.

No toda deuda bloquea H6. Sí bloquean: click-through, controles críticos inaccesibles, clipping que impide Golden Path, save feedback falso, modales no exclusivos, acción destructiva insegura, input duplicado o build no operable. La estética fina puede diferirse si la interfaz es representativa y coherente.

118. Criterios de Sprint 16

Sprint 16 requiere que StoreInitial y la presentación representativa permitan completar el flujo sin contradicciones. Para UI: Operations abre sin mover jugador; HUD y feedback son legibles sobre la nueva escena; overlays no ocultan puntos críticos; audio/UI feedback está conectado; no quedan placeholders visibles no exceptuados; y los paneles no muestran datos incompatibles con el estado.

La suite previa 1215 EditMode + 70 PlayMode es baseline técnica, no aprobación visual. Deben añadirse recorrido manual, capturas, Player.log y build post-integración.

119. Criterios de Sprint 17

Sprint 17 estabiliza UX: foco, navegación, click-through, resoluciones, escalas, localización mínima, accesibilidad, tutorial, Golden Path, campaña de siete días, performance y regresión. No debe abrirse para añadir sistemas mayores.

La Definition of Done UI exige cero S0/S1, S2 resueltos o aceptados con riesgo, pruebas P0/P1, build externa, evidencia y actualización de producción/trazabilidad.

120. Criterios de H6

H6 exige una experiencia coherente en build: inicio, slots, StoreInitial, operaciones principales, checkout, cierre, save/load, feedback, accesibilidad base y salida. UI no puede depender de debug tooling inaccesible ni presentar una capa de Sprint explícita como producto final.

Debe existir una decisión formal sobre qué placeholders se aceptan. La UI puede ser representativa, pero no puede ser engañosa, insegura o bloquear el Golden Path. ES/EN, opciones y soporte de resolución deben coincidir con el alcance declarado para la build.

121. Playbook: revisión de una pantalla

1. identificar requisito y estado; 2) abrir en resolución mínima; 3) comprobar jerarquía sin interactuar; 4) recorrer con ratón; 5) recorrer con teclado; 6) probar 80/150 %; 7) pseudo-localizar; 8) provocar vacíos, errores y loading; 9) validar audio/feedback;

10) capturar evidencia.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

122. Playbook: creación de un componente

Definir rol, props, estados, tokens, navegación, localización, accesibilidad y errores antes de authoring. Implementar en fuente compartida, añadir sandbox/TestLab, tests de estado y capturas. No copiar y modificar un botón existente sin registrar variante.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

123. Playbook: cerrar click-through

Reproducir con logging de frame y action maps; comprobar EventSystem; añadir pointer-over-UI/consumo; cubrir press y release; verificar Store y StoreInitial; probar panel, modal y cierre; ejecutar suite; generar build; archivar video y Player.log.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

124. Playbook: integrar localización

Extraer strings; definir keys y variables; crear tablas ES/EN; reemplazar concatenaciones; añadir selector/fallback; pseudo-localizar; probar 1280×720 y 150 %; revisar terminología; ejecutar build; registrar clipping.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

125. Playbook: revisión de accesibilidad

Probar sin audio, con color simulado, teclado sin ratón, texto 150 %, UI 150 %, reduced motion y confirmaciones activas/desactivadas. Registrar tarea imposible, pérdida de foco, target pequeño, mensaje fugaz o señal solo cromática.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

126. Playbook: migrar UI técnica

Inventariar canvas serializado y runtime; marcar dependencias; crear reemplazo compartido; mantener contrato; desactivar legado tras prueba; validar input y escena; eliminar solo cuando no exista rollback necesario; documentar sustitución.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

127. Playbook: añadir icono o fuente

Verificar licencia y procedencia; importar con settings; registrar catálogo; añadir fallback; probar escalas; comprobar caracteres/atlas; actualizar legal; evitar subir archivos de fuente fuera del repositorio autorizado.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

128. Playbook: aprobar un modal

Abrir desde teclado y ratón; verificar foco seguro; bloquear mundo; probar Cancel; probar confirmación una vez; provocar fallo de operación; restaurar foco; cambiar escena con modal abierto; comprobar que no queda overlay huérfano.

El resultado se registra en QA con build/commit, evidencia, defectos y decisión. Un playbook no sustituye criterios específicos del sistema.

129. Antipatrones y elementos prohibidos

Se prohíbe: color como única señal; texto crítico menor de 12 px efectivo; botones sin verbo; modal sin cancelación; click-through; doble submit; acción destructiva como foco inicial; números sin unidad; strings finales hardcodeados; concatenación localizable; controles invisibles navegables; scroll sin indicador; tooltip obligatorio sin teclado; hover como única interacción; fuente sin licencia; icono copiado de marca; panel de debug presentado como UI final; **PASS** basado solo en captura conceptual.

También se evita crear un Canvas por mensaje sin jerarquía, multiplicar fábricas de componentes, usar **Find** por label visible, almacenar estado funcional en componentes visuales o mutar dominio durante render/proyección.

130. Riesgos y decisiones de rollback

Los riesgos principales son regresión de input, foco roto, pérdida de save por confirmación ambigua, clipping a 720p, inconsistencia entre Store y StoreInitial, proliferación de estilos, localización tardía y reescritura prematura. Cada cambio UI debe poder desactivarse o revertirse sin alterar el dominio.

El rollback conserva la pantalla anterior, feature flag o commit identificable hasta pasar targeted suite y build. No mantener dos UI activas indefinidamente. Al retirar legado se actualizan escenas, tests, Binder, producción y trazabilidad.

131. Glosario

HUD: información persistente de alta frecuencia. **Overlay:** capa sobre mundo. **Modal:** capa que requiere decisión antes de continuar. **Toast:** mensaje temporal no bloqueante. **Focus:** control activo para teclado/mando. **Hover:** puntero sobre control. **Input exclusivity:** una capa UI impide que el mismo evento llegue al mundo. **Token:** valor semántico reusable. **Projection:** representación de estado sin mutación. **Placeholder:** solución temporal explícita. **Golden Path:** recorrido obligatorio del vertical slice. **H6:** gate del vertical slice funcional y presentable.

132. Historial de revisiones y mantenimiento

Esta versión consolida v0.3, reedición v0.4, v0.4, v0.5, UX históricos, localization, catálogos, ADR de input/UI, Sprint 16 y código/escenas reales. La próxima revisión debe ocurrir tras cerrar click-through, integrar StoreInitial, decidir localización H6 o sustituir las fábricas runtime.

Cada revisión registra versión, fecha, autor, cambios, fuentes, decisiones sustituidas y hash. No se edita una baseline congelada; se publica nueva versión y se conserva la anterior. La auditoría global se repetirá al terminar la generación de documentos especializados.

133. Estado de slot vacío

Un slot vacío presenta identidad estable, ausencia de fecha y acción `New Game`. No debe parecer corrupto ni bloqueado. Al crear partida, la UI debe deshabilitar el control hasta conocer resultado, evitar doble creación y seleccionar la sesión creada. Si la operación falla, el slot sigue vacío y muestra causa recuperable. El tutorial y autosave markers se inicializan una sola vez. La tarjeta debe mantener espacio para estados más largos en ES/EN y para icono de estado futuro.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

134. Estado de slot válido

Un slot válido muestra día, saldo, fecha de actualización y acción `Continue`; las acciones Replace/Delete son secundarias o están tras menú. La UI no reconstruye información desde strings: consume `SlotDescriptor`. Continuar debe cargar exactamente el slot seleccionado, impedir doble navegación y conservar foco si la carga falla. La fecha UTC actual es diagnóstico; la versión final debe usar locale o indicar zona de manera comprensible.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

135. Slot recuperado desde backup

Cuando la lectura primaria falla y se usa backup válido, la tarjeta informa `Recovered from backup` de forma persistente durante la sesión de selección. No se trata como éxito silencioso. El jugador debe poder continuar sin miedo, pero el sistema registra qué generación se recuperó. Un banner o estado de tarjeta es preferible a un toast efímero. La siguiente escritura no debe destruir el único backup válido sin estrategia de recuperación.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

136. Slot con schema no soportado

El estado incompatible bloquea Continue y explica que la versión actual no puede abrir la partida. No se ofrece Delete como única salida sin confirmación clara. La UI debe diferenciar schema futuro, schema demasiado antiguo y datos corruptos cuando la infraestructura pueda hacerlo. El mensaje técnico puede incluir versión en detalle expandible, pero la acción principal es actualizar, migrar o volver, no experimentar con el archivo.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

137. Slot corrupto sin backup

La corrupción sin backup es un error grave. La tarjeta no muestra saldo o día inventados. Debe ofrecer información sobre ubicación de save o exportación solo cuando exista tooling seguro; nunca sugerir que New Game sobrescribirá automáticamente el archivo. Delete permanece destructivo y confirmado. QA prueba archivos truncados, JSON inválido y backup inválido.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

138. Almacenamiento no disponible

Permisos, ruta inaccesible o fallo de IO se presentan como problema de almacenamiento, no como slot vacío. Crear, continuar o borrar puede deshabilitarse. El mensaje indica reintento y recomienda revisar permisos/espacio sin culpar al jugador. La UI no entra en bucle de modal. Player.log conserva detalle técnico y la pantalla sigue operable para Quit o Help.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

139. Overlay sin sesión activa

Si Store carga sin slot activo, `StoreHudScreen` muestra un overlay que dirige a MainMenu. Debe bloquear mundo y evitar que una partida fantasma avance. La acción `Return to Main Menu` es segura porque no existe sesión válida. Esta condición debe ser rara y quedar en logs; si ocurre por carrera de inicialización, se trata como defecto, no como flujo habitual.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

140. HUD en BeforeOpen

Antes de abrir, HUD muestra día, estado, saldo y save, pero clientes/cola permanecen cero o con señal neutral. La acción de apertura vive en DayCycle/Operations. No se debe usar color de éxito para un estado todavía cerrado. El tutorial puede señalar el panel correspondiente sin bloquear movimiento salvo durante la burbuja interactiva.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

141. HUD durante Open

Durante Open, los valores de clientes y cola se actualizan a intervalo controlado. No se reconstruye todo el Canvas cada frame. Alertas se agregan: stock, cola y cierre no deben producir una cascada de mensajes. El estado Open se acompaña de texto. Cash cambia después de transacción confirmada y el save status no promete persistencia continua.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

142. HUD durante Closing y Closed

Closing debe indicar que no se aceptan determinadas operaciones y qué falta para cerrar. Closed activa el checkpoint/autosave coordinado una vez. La UI diferencia `Closed`, `Saving`, `Saved`, `Save failed` y `No autosave required`. Continue to Next Day solo aparece cuando el estado y autosave permiten avanzar según contrato.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

143. Fila de panel de gestión

La fila actual contiene Label, Value y StatusText con proporciones 2:1:2. La versión consolidada define alineación, unidad, wrap y estado vacío. Label identifica objeto; Value muestra dato principal; Status explica estado o consecuencia. No introducir botones invisibles dentro de toda la fila sin foco explícito. Las filas de 58 px deben crecer con text scale y localización.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

144. Panel Help

Help resume navegación, activación, cancelación, paneles y autosave. Debe derivarse de bindings reales y actualizarse si cambian. No afirmar soporte de gamepad, touch o remapeo no validado. La ayuda se puede abrir desde MainMenu y pausa, permanece legible sin simulación activa y enlaza a tutorial reinicializable cuando proceda.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

145. Burbuja de tutorial

La burbuja contiene título, cuerpo, anchor semántico, Next y Skip. El anchor no debe ser un nombre de GameObject frágil sin fallback. Si el elemento objetivo no existe, la burbuja explica el paso sin quedar fuera de pantalla. Skip respeta confirmaciones destructivas configurables; Restart reinicia progreso del slot, no ajustes globales.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

146. Modal de confirmación

El modal se compone de backdrop bloqueante, panel, título, cuerpo, confirmación y cancelación. El fondo captura raycast. Cancel/Escape cierra solo el modal. La confirmación llama una vez al comando, espera resultado y no permite interacción detrás. La acción segura recibe foco inicial; la peligrosa usa estilo y texto específicos.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

147. Operations: Guide

Guide traduce el procedimiento técnico a pasos observables: instrucción, paso actual, saldo, ventas completadas y estado. Mientras siga siendo tooling interno puede mencionar Phase 2, pero una build externa destinada a usuarios debe reemplazar lenguaje de sprint por objetivo jugable. La condición Completed se deriva del servicio, no de clicks de UI.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

148. Operations: Shop

Shop lista muebles comprables y productos por caja. Cada entrada muestra nombre, footprint/case, coste y acción. La versión representativa ordena una unidad/caja; una futura cantidad utiliza stepper y coste total. Las acciones fallidas muestran motivo y no descuentan dinero. El catálogo debe scrollar y mantener foco tras rebuild.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

149. Operations: Delivery

Delivery separa pedidos pendientes, entregas disponibles y recepción. El botón Receive actúa sobre un ID estable. Tras recibir, la fila se actualiza sin duplicar unidades. Estado vacío indica que se necesita realizar pedido y ofrece ruta a Shop. La llegada puede tener feedback visual/sonoro, pero la acción de recepción sigue siendo explícita.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

150. Operations: Stock

Stock resume almacén y contenido. Debe distinguir muebles entregados, productos por unidad, capacidad y ubicaciones. La UI representativa puede ser textual; la futura superficie ofrece filtros sin perder el total. Toda mutación de inventario usa servicios atómicos; una fila no edita directamente snapshots.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

151. Operations: Displays

Displays lista fixtures, asignaciones y stock. Acciones de colocar, asignar o reponer se habilitan según estado. Debe quedar claro si una acción afecta catálogo, placement o inventario. El mismo producto no se identifica solo por color o icono. Los displays físicos en StoreInitial y su fila deben compartir ID trazable.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

152. Operations: Customer

Customer muestra actividad suficiente para probar el slice. El lenguaje final evita exponer estados internos crudos cuando no aportan decisión. Paciencia usa texto/indicador con umbral, no solo barra verde-roja. El panel no sustituye señales en el mundo. Para QA puede existir un modo diagnóstico separado y marcado.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

153. Operations: Settings

Settings de Sprint 16 incluye canales de audio y controles representativos. Debe coexistir con Accessibility sin duplicar preferencias. Los pasos 0/50/100 son útiles para validar routing, pero no equivalen al menú final. El valor persiste, la etiqueta usa porcentaje y el audio de preview no se reproduce en bucle al navegar.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

154. Feedback canvas de Sprint 16

`Phase1FeedbackPresenter` crea panel y texto temporales en un Canvas separado. La guía exige integrarlo con message duration, reduce motion, prioridad, deduplicación y stack de overlays. Los colores se acompañan de texto. Mensajes de venta, pedido, error y save no se pisan. El panel no captura input si es pasivo, pero tampoco debe impedir leer controles.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

155. Canvases técnicos serializados

MainMenu, Store y StoreInitial contienen EventSystem, InputSystemUIInputModule y canvases técnicos. El overlay runtime no elimina esos controles. Durante la migración, se debe verificar que no reciben foco o click detrás, que no duplican navegación y que sus sorting orders no compiten. La retirada exige una prueba de scene flow y rollback documentado.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

156. Ciclo de vida entre escenas

El composition root instala pantallas al cargar escena y destruye instancias runtime anteriores por nombre. La guía exige idempotencia, ausencia de duplicados y limpieza de suscripciones. Cambiar escena con modal abierto no deja InputGate en UI ni EventSystem con foco destruido. StoreInitial debe incorporarse explícitamente cuando se convierta en escena objetivo.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

157. Refresco, rebuild y rendimiento

HUD puede refrescar valores con intervalo; paneles se reconstruyen al abrir o cambiar snapshot. Rebuild dinámico no debe generar GC perceptible, perder scroll/foco ni duplicar listeners. Profiling registra allocations, Canvas rebuild y spikes en listas. Para H6, el tamaño de datos es pequeño, pero el patrón debe evitar crecimiento no acotado.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

158. Persistencia de preferencias

Accesibilidad, tutorial y autosave markers se almacenan mediante repositorios JSON atómicos. Las preferencias globales no deben quedar ligadas accidentalmente a un slot; el progreso tutorial sí es por slot según diseño. Los fallos de escritura muestran estado sin bloquear el juego. Las claves y paths se documentan para soporte y migración.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

159. Taxonomía de claves de localización

Las claves usan dominios: `ui.main_menu`, `ui.slot`, `ui.hud`, `ui.operations`, `ui.inventory`, `ui.order`, `ui.display`, `ui.customer`, `ui.checkout`, `ui.economy`, `ui.day`, `ui.pause`, `ui.settings`, `ui.accessibility`, `ui.tutorial`, `notification`, `error`. Variables se nombran y formatean. No se reutiliza una key por coincidencia textual si la intención difiere.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

160. Interfaz de sistemas futuros

Publishing, desarrollo interno, ecommerce, plataforma, infraestructura y mercado conservan patrones históricos: dashboards, hitos, contratos, catálogos, capacidad, incidentes y comparativas. Permanecen como visión. Cuando se abran, cada uno requiere UX Flow, requisitos, modelo de datos y prototipo; no se implementan copiando las pantallas extensas de v0.3.

La revisión se considera completa solo cuando existe evidencia de estado nominal, vacío, bloqueado y error, además de navegación y localización cuando apliquen.

161. Matriz de resoluciones, escalas e idiomas

La matriz define la campaña mínima. Los casos adicionales se añaden según riesgo, hardware y cambios de layout.

Resolución	Aspecto	UI scale	Text scale	Idioma	Entrada	Objetivo
1280×720	16:9	100%	100%	ES/EN o pseudo	Ratón + teclado	Resolución mínima; clipping y scroll
1280×720	16:9	150%	150%	Pseudo	Teclado	Peor caso de expansión y foco
1920×1080	16:9	80%	80%	ES	Ratón	Targets y legibilidad mínima
1920×1080	16:9	100%	100%	ES/EN	Ratón + teclado	Referencia
1920×1080	16:9	150%	150%	EN/pseudo	Teclado	Accesibilidad
2560×1440	16:9	100%	100%	ES	Ratón	Escala física y espacios
3440×1440	21:9	100%	100%	EN	Ratón + teclado	Anclaje y longitud de línea
1024×768	4:3	100%	100%	ES	Ratón	Deuda/configuración actual; detectar bloqueo

162. Matriz de capas y bloqueo de input

Cada capa define si captura puntero, si cambia action map, si pausa y dónde restaura foco.

Capa	Captura puntero	UI exclusiva	Pausa simulación	Cancel	Foco al cerrar
HUD pasivo	No salvo botones	No	No	Abre pausa si no hay overlay	N/A
Management panel	Sí	Sí	Según diseño; actualmente gestión bloqueada	Cierra panel	Botón que abrió
Operations	Sí	Sí	No necesariamente; gameplay input bloqueado	Cierra	OpenOperations
Tutorial interactivo	Sí en burbuja	Sí mientras requiere decisión	No	Según paso/Skip	Control anterior
Confirmation	Sí, backdrop completo	Sí	Hereda capa	Cancela	Control solicitante

Capa	Captura puntero	UI exclusiva	Pausa simulación	Cancel	Foco al cerrar
Pause menu	Sí	Sí	Sí o estado gobernado	Resume	Primer HUD seguro
Feedback pasivo	No	No	No	No consume	N/A

163. Definition of Ready por tipo de trabajo UI

No se inicia un cambio sin requisitos y datos suficientes. Esto reduce iteraciones y evita que la vista invente contratos.

Tipo	Ready mínimo
Pantalla nueva	UX Flow, datos/proyección, estados nominal/vacío/error, navegación, localización, owner y gate
Componente	Rol, tokens, estados, props, foco, target, texto y tests
Cambio de input	Mapa/contexto, prioridad, dispositivos, casos de propagación y PlayMode plan
Localización	Keys, variables, tabla fuente, terminología y pseudo-localización
Accesibilidad	Necesidad, rango/default, persistencia, efecto visual y QA
Migración de UI	Inventario legado, dependencias, rollback, equivalencia y criterio de retirada
Asset icono/fuente	Licencia, catálogo, import settings, variantes y fallback

164. Definition of Done por tipo de trabajo UI

Done separa implementación, validación y evidencia.

Tipo	Done mínimo
Pantalla nueva	Flujo completo, estados cubiertos, ratón/teclado, 720p/1080p, escalas, no click-through, tests, evidencia
Componente	Fuente compartida, estados, foco, localización, accesibilidad, sandbox y catálogo
Cambio de input	Targeted + PlayMode + build; evento consumido una vez; contexto restaurado
Localización	ES/EN o alcance aprobado, fallback, pseudo, no truncado crítico y build
Accesibilidad	Persistencia, aplicación a todas las capas, combinaciones extremas y documentación
Migración de UI	Legado desactivado/retirado, no duplicados, rollback cerrado, escenas/tests actualizados

Tipo	Done mínimo
Asset icono/fuente	Licencia registrada, import validado, fallback, escalas y créditos

165. Work packages recomendados

Los paquetes son una propuesta de ejecución; deben incorporarse a producción y trazabilidad antes de considerarse compromiso.

ID	Objetivo	Prioridad/Periodo	Trazabilidad
UI-WP-01	Cerrar click-through de Operations y cualquier HUD interactivo	P0 / Sprint 16	VS-INP-001, UX-INP-001...006
UI-WP-02	Añadir PlayMode tests del frame de apertura, doble click y modal	P0 / Sprint 16	Input exclusivity
UI-WP-03	Integrar StoreInitial en composition root/settings sin perder Store fallback	P0 / Sprint 16	Scene migration
UI-WP-04	Auditar y desactivar canvases técnicos detrás de overlays	P1 / Sprint 16	Duplicate UI/focus
UI-WP-05	Unificar Sprint15UiFactory y Phase1RuntimeUiFactory mediante tokens	P1 / Sprint 17	Consistency/debt
UI-WP-06	Crear UiTheme/typography/motion assets o equivalente	P1 / Sprint 17	Design system
UI-WP-07	Sustituir LegacyRuntime.ttf por fuente licenciada o decisión formal	P1 / pre-H6	Typography/legal
UI-WP-08	Crear catálogo de iconos y fallbacks; registrar seis product icons	P2 / Sprint 17	Iconography
UI-WP-09	Integrar localization tables ES/EN y pseudo-localización	P1 / Sprint 17/H6	VS-UI-004, UX-LOC-001
UI-WP-10	Exponer message duration y completar ajustes de accesibilidad	P1 / Sprint 17	UX-ACC
UI-WP-11	Reemplazar audio steps por controles finales o declarar placeholder H6	P2 / Sprint 17	Audio settings
UI-WP-12	Normalizar foco, restore y navegación de tabs/listas	P1 / Sprint 17	Keyboard/accessibility
UI-WP-13	Integrar feedback canvas con toast/message service y prioridades	P2 / Sprint 17	Feedback consistency
UI-WP-14	Ejecutar matriz 720p/1080p/1440p, 80/100/150 y teclado	P0 / H6	UI QA gate
UI-WP-15	Eliminar lenguaje de sprint/debug de candidata pública	P1 / H6	Presentation
UI-WP-16	Documentar y probar opciones visuales/resolución reales	P2 / post-H6 si no comprometido	Settings

166. Protocolo de revisión semanal de UI

Una vez por semana durante integración, revisar: defectos S0-S2; cambios de escenas; nuevos strings hardcodeados; nuevos colores literales; componentes duplicados; capturas a 720p; foco por teclado; click-through; localización; licencias; Player.log; y diferencias entre Store y StoreInitial. El resultado se registra como checkpoint, no como build PASS automática.

La revisión debe comparar código y experiencia. Un cambio de layout que no modifica lógica puede introducir un bloqueo real; un cambio de dominio puede dejar labels obsoletos. El checklist se cierra con acciones y owner, no con observaciones genéricas.

167. Protocolo de regresión tras cambiar datos o dominio

Cuando cambia un snapshot, estado, enum o servicio, revisar toda proyección que lo presenta. Añadir estado desconocido/fallback; actualizar localización; probar save antiguo; comprobar panel nominal/vacío/error; y confirmar que la UI no muta el modelo para adaptarlo. Los cambios de money, quantity, IDs o day states requieren especial atención.

La regresión incluye MainMenu si afecta slots, HUD si afecta resumen, Operations si afecta procedimiento y QA traceability. No aceptar una compilación verde como prueba de semántica correcta.

168. Protocolo de captura y referencia visual

Las capturas de referencia se generan desde la build o Editor identificado, con resolución y escala. Se guardan versiones nominal, hover/focus, modal, error, vacío y escala alta. Los conceptos se almacenan separados y etiquetados. Una captura aprobada no fija píxeles eternamente: fija jerarquía, legibilidad y lenguaje, salvo que se declare pixel-perfect.

Para StoreInitial, las capturas incluyen diferentes cámaras y fondos para verificar contraste. No se utilizan capturas retocadas como evidencia de runtime.

169. Trazabilidad mínima de una pantalla

Cada pantalla debe enlazar: propósito de GDD/UX; Requirement IDs; datos/proyección; comandos; ADR técnica si aplica; assets y tokens; casos de prueba; ejecuciones; build; defectos; evidencia; y versión documental. Esta cadena permite saber si una pantalla existe, funciona, se ha visto en build y está aceptada.

La matriz actual no debe marcar cobertura completa solo por tener un Test ID. La cobertura UI completa exige al menos implementación real, caso diseñado, ejecución atribuible y evidencia adecuada al riesgo.

170. Perfil técnico: Sprint15RuntimeCompositionRoot

Es el composition root de la UI de Sprint 15. Construye servicios de slots, sesión, accesibilidad, tutorial y autosave; escucha `SceneManager.sceneLoaded`; y crea pantallas según nombres configurados. Su diseño preserva escenas existentes, pero depende de `_mainMenuSceneName` y `_storeSceneName`, todavía con `Store`. Debe reconocer `StoreInitial` cuando se apruebe la migración. La instalación debe ser idempotente, sus servicios deben sobrevivir correctamente entre escenas y la destrucción debe restaurar `InputGate`. La UI no debe buscar servicios globales adicionales fuera de este root sin ADR.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

171. Perfil técnico: MainMenuSlotScreen

Construye un Canvas runtime, fondo, contenido, tarjetas de slot, footer y overlays. Se suscribe a cambios de accesibilidad y Cancel. En Initialize entra en UI exclusiva. Sus responsabilidades actuales incluyen demasiada presentación y wiring; una evolución separaría view, controller y factory sin reescribir servicios. Debe protegerse contra rebuild mientras una operación está activa, restaurar foco, localizar todos los textos y evitar que el Canvas serializado inferior reciba input. La pantalla es funcional y forma parte del Golden Path.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

172. Perfil técnico: StoreHudScreen

Construye HUD, navegación, paneles, tutorial, pausa y confirmaciones. Se suscribe a snapshot, autosave, accesibilidad y Cancel. Al inicializar sale de UI exclusiva para permitir gameplay; al abrir capas vuelve a entrar. La clase mezcla render, navegación, comandos y tutorial, por lo que tiene riesgo de regresión durante cambios. Antes de refactorizar se necesitan tests de equivalencia. El refresh periódico no debe alterar foco ni recrear controles innecesariamente.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

173. Perfil técnico: Sprint15UiFactory

Centraliza Canvas, Panel, Text, Button, layouts y selección. Usa `LegacyRuntime.ttf`, `Navigation.Automatic`, colores literales y tamaños fijos. Es una base útil pero no un sistema de diseño completo. Debe evolucionar hacia tokens y variantes semánticas. `EnsureEventSystem` solo crea EventSystem si no existe; las escenas actuales aportan `InputSystemUIInputModule`. Si se carga una escena sin módulo, crear solo EventSystem no basta para input completo, por lo que el bootstrap debe validar módulo o depender explícitamente de escena.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

174. Perfil técnico: StoreUiProjectionService

Convierte `IntegratedGameStateSnapshot` en HUD y filas de panel. Esta separación es correcta: la vista no inspecciona directamente todo el dominio. Debe mantenerse pura y localizada mediante claves/formatters, no strings ingleses. Los estados desconocidos necesitan fallback visible y logging. El servicio puede generar ViewModels con IDs semánticos, severidad e icono, evitando que la vista infiera estado desde texto. Los tests de proyección son parte de la regresión de datos.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

175. Perfil técnico: UiNavigationState

Modela una pila de entradas con layer y focus target. Evita duplicar la misma entrada superior. Debe convertirse en la autoridad de capas runtime o quedar claramente como modelo de referencia; la navegación manual paralela aumenta riesgo. Una entrada futura puede incluir owner, close policy, blocksWorld y previous selection. No almacenar referencias Unity en dominio. Las pruebas deben cubrir push/pop, duplicate, clear, invalid target y restauración tras destrucción.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

176. Perfil técnico: AccessibilitySettingsService

Expone ajustes actuales, valida mediante `UiAccessibilitySettings`, persiste en repositorio y emite Changed. La UI debe tratar un fallo de persistencia sin descartar el cambio visual silenciosamente: mostrar estado y decidir rollback o retry. Los defaults son parte de producto y deben versionarse. Cambiar límites 80–150 o duración 1–30 requiere revisión UX, tests y migración de archivos existentes.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

177. Perfil técnico: TutorialService

Genera burbujas desde progreso persistido. Sus textos y anchors están hardcoded. Debe migrar a keys y contenido data-driven cuando se establezca el tutorial. Los pasos actuales cubren el vertical slice y son suficientes como baseline. La UI debe validar que avanzar no dependa solo de pulsar Next cuando el diseño exige acción real. Skip y Restart tienen consecuencias por slot y necesitan copy inequívoco.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

178. Perfil técnico: Sprint15InputSystemBridge

Instala un objeto persistente antes de escenas, crea `ProjectInputActions`, habilita UI y convierte Cancel en señal global. Actualmente no enruta Navigate/Submit de forma propia porque InputSystemUIInputModule de escena lo hace. Esta división debe documentarse. Si se elimina el EventSystem serializado, el bridge debe asumir o instalar el módulo correcto. No habilitar mapas duplicados que generen dos Submit. Las pruebas deben inspeccionar número de módulos y callbacks.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

179. Perfil técnico: Sprint15InputMapGate

Guarda el contexto previo y cambia a `InputContextId.UI`. Es idempotente para Enter y restaura al salir. Debe manejar scene unload, root destruido, contexto previo inválido y capas anidadas. El booleano actual no modela múltiples overlays; cerrar una capa podría restaurar Gameplay aunque otra siga abierta si los callers no coordinan. La evolución recomendada es ref count o stack de tokens de exclusividad, con tests de anidamiento.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

180. Perfil técnico: Phase1OperationsScreen

Es una UI representativa de Sprint 16 con siete tabs, catálogo y acciones. Usa un Canvas de sorting 4550, por debajo de Sprint15 HUD 5000, y entra en UI exclusiva al abrir. El botón de apertura permanece visible con Gameplay activo y es el punto del defecto click-through. El contenido se destruye/recrea por tab. Debe conservar selección, scroll y foco, y separar lenguaje de QA del producto antes de distribución externa.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

181. Perfil técnico: Phase1RuntimeUiFactory

Duplica funciones de Sprint15UiFactory con otra paleta y alturas. No garantiza EventSystem; solo selecciona si existe. Su ScrollRect usa Mask e Image como viewport. La duplicación permitió autonomía de Phase 1, pero ya es deuda de consistencia. La migración debe comparar comportamientos y elegir una fuente común sin romper Operations. Los valores de sorting, padding y font size se convertirán en tokens o presets.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

182. Perfil técnico: Phase1FeedbackPresenter

Recibe feedback, resuelve color y evento de audio, muestra panel/texto temporal y dispara VFX. La UI debe garantizar que el feedback se emite después de operación confirmada. El presenter no debe decidir reglas de negocio. Su canvas necesita coordinación con mensajes de StoreHud y preferencias de duración. Los mensajes de error crítico deben escalar a modal/banner en lugar de desaparecer. El texto final se localiza y la severidad no se deduce solo del color.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

183. Perfil técnico: escenas MainMenu, Store y StoreInitial

MainMenu y las dos tiendas contienen EventSystem con InputSystemUIInputModule y Canvas técnico. StoreInitial es actualmente copia técnica de Store y conserva ReturnButton/ScopeNotice. La UI runtime se instala por nombre de escena configurado, todavía Store. La integración de StoreInitial debe actualizar settings, build profile, composition root y pruebas. Antes de eliminar los canvases técnicos se verifica que no contienen la única ruta de regreso o fallback de emergencia.

Estado de aprobación: el perfil describe la evidencia observada; cualquier cambio estructural requiere targeted tests, PlayMode y revisión de build.

184. Inventario de assets UI observados

El inventario demuestra qué existe y qué permanece vacío. Las carpetas no se interpretan como contenido.

Categoría	Cantidad/estado	Detalle	Interpretación
UXML	0	No se encontraron archivos <code>.uxml</code>	UI Toolkit no implementado
USS	0	No se encontraron archivos <code>.uss</code>	No hay stylesheet UI Toolkit
Fuentes de proyecto	0	<code>Assets/_Project/UI/Fonts/.gitkeep</code>	Tipografía final pendiente
Iconos UI generales	0	<code>Assets/_Project/UI/Icons/.gitkeep</code>	Sistema de iconos pendiente
Iconos de producto	6 PNG	Memory Core, Cloud Runner, Vertex One, Orbit Pad, Neon Drift, Signal Pro	Contenido de catálogo; no sistema general

Categoría	Cantidad/estado	Detalle	Interpretación
Audio UI	2 WAV	UiConfirm, UiError	Feedback representativo
Input actions	1 asset	Mapas Player y UI	Implementado
Canvases de escena	MainMenu/Store/StoreInitial	Canvas + EventSystem/InputSystemUIInputModule	Legado técnico activo
Fábricas runtime	2	Sprint15UiFactory, Phase1RuntimeUiFactory	Duplicación pendiente de consolidación

185. Copy de éxito

Confirmar qué ocurrió y, si importa, el resultado: `Order received · 12 units added to Warehouse`. No usar `Success` aislado. No mostrar antes del commit. Evitar exclamaciones repetitivas.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

186. Copy de advertencia

Describir riesgo y decisión: `Low stock · 2 units remain`. La advertencia no bloquea si la operación es válida. Ámbar, icono y texto. Debe poder agregarse y no repetirse cada frame.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

187. Copy de error recuperable

Estructura: qué falló, por qué si es seguro afirmarlo, y siguiente acción. Ejemplo: `Order could not be placed · Not enough cash. Reduce quantity or close.` No mostrar excepción como cuerpo principal.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

188. Copy de error de datos

No minimizar corrupción o schema. Ejemplo: `This save cannot be opened by this version. The file was not changed.`

Añadir detalle técnico opcional, backup y soporte. No ofrecer sobrescritura automática.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

189. Copy destructivo

Título con verbo/objeto; cuerpo con alcance irreversible; botones específicos. `Delete Slot 2? / Deletes the primary save, backup, tutorial progress and autosave marker. / Delete + Keep Slot.`

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

190. Copy de bloqueo

Explicar condición: `Complete Close is unavailable · 1 checkout transaction is still active.` Si existe acción, enlazarla. No usar `Invalid state` salvo en detalle técnico.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

191. Copy de loading

Usar presente: `Loading StoreInitial...`, `Saving Slot 1...`. No prometer duración. Si tarda, permitir cancelación solo cuando sea segura. Al completar, reemplazar por estado real.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

192. Copy de vacío

Evitar tono de error: `No supplier orders yet. Open Shop to place the first order.` El estado vacío puede ser normal y educativo.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

193. Copy de tutorial

Una idea y una acción por burbuja. Verbo al inicio. No describir todo el sistema. Permitir cerrar, saltar y consultar ayuda. El tutorial no debe cubrir controles relevantes.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

194. Copy de tooling interno

Etiquetas como Sprint, Phase, debug, escenario o QA deben llevar marca `DEV` y no entrar en candidata pública. La guía interna puede conservarlas; la build externa debe usar objetivo jugable.

Todos los textos finales se implementan mediante localization keys, variables tipadas y revisión ES/EN. El tono se valida junto al estado funcional.

195. Extracción de strings

Inventariar literales visibles en MainMenuSlotScreen, StoreHudScreen, StoreUiProjectionService, TutorialService, Phase1OperationsScreen y FeedbackPresenter. Excluir nombres técnicos no visibles. Cada literal recibe contexto, screenshot y owner.

La localización no se difiere hasta el final del layout: sus restricciones deben probarse antes de aprobar componentes y pantallas.

196. Diseño de keys

Crear keys estables por intención, no por texto. Separar títulos, labels, mensajes y botones aunque coincidan. Variables usan nombres semánticos: `{slotNumber}`, `{cash}`, `{day}`, `{units}`.

La localización no se difiere hasta el final del layout: sus restricciones deben probarse antes de aprobar componentes y pantallas.

197. Tablas fuente ES/EN

Español puede ser fuente de diseño; inglés se revisa comercialmente. Definir locale fallback. No guardar IDs traducidos. Los estados del dominio se mapean a keys y no se muestran crudos.

La localización no se difiere hasta el final del layout: sus restricciones deben probarse antes de aprobar componentes y pantallas.

198. Pseudolocalización

Expandir al menos 30 %, añadir diacríticos, envolver delimitadores y conservar variables. Ejecutar MainMenu, HUD, todos los paneles, Operations, tutorial, modales y errores. Detectar hardcodeo residual.

La localización no se difiere hasta el final del layout: sus restricciones deben probarse antes de aprobar componentes y pantallas.

199. Formatos culturales

Dinero, fecha, hora, separadores y plural usan locale. `EUR 1000` no se concatena manualmente. El formato de fecha de slot se revisa con zona. Unidades se localizan y mantienen consistencia.

La localización no se difiere hasta el final del layout: sus restricciones deben probarse antes de aprobar componentes y pantallas.

200. QA lingüística

Revisar terminología, variables, orden, truncado, fallback, caracteres, line breaks, iconos con texto y screenshots. Un PASS lingüístico identifica build e idioma. La calidad de traducción y la integridad funcional son resultados separados.

La localización no se difiere hasta el final del layout: sus restricciones deben probarse antes de aprobar componentes y pantallas.

201. Campaña QA: foco completo

Recorrer toda la UI usando solo teclado. Registrar orden, indicador, scroll, retorno, modales, tabs y rebuild. Empezar en MainMenu sin ratón; crear/cargar slot; abrir paneles; completar cierre; volver. Repetir tras scale 150 %.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

202. Campaña QA: propagación de puntero

Hacer click rápido, click sostenido, release fuera, doble click y alternancia UI/suelo sobre cada botón cercano al mundo. Grabar posición del jugador/ghost antes y después. Probar Operations, HUD, panel, modal, tutorial, pause y scroll.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

203. Campaña QA: slots y recovery

Preparar slot vacío, válido, backup recovery, schema incompatible, primary corrupt, ambos corruptos y storage denied. Verificar copy, acciones, ausencia de pérdida y logs. Repetir create/delete/replace con confirmaciones desactivadas y activadas.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

204. Campaña QA: escalado extremo

UI 150 + text 150 a 1280×720; UI 80 + text 80 a 1080p; combinar 150/80. Revisar targets, clipping, paneles scrollables, modal, tutorial y footer. No aceptar elementos inaccesibles aunque el texto sea legible.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

205. Campaña QA: localización

ES, EN y pseudo. Revisar strings, variables, moneda, fechas, plural, tabs, botones y errores. Buscar literales ingleses residuales. Capturar mayor expansión y reparar layouts, no abreviar sin decisión.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

206. Campaña QA: accesibilidad sin audio/color

Silenciar todos los canales; usar simulador de daltonismo; navegar por teclado; activar reduced motion; aumentar duración. Completar Golden Path. Todo evento crítico debe seguir siendo comprensible.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

207. Campaña QA: cambio de escena y overlays

Abrir panel/modal/tutorial/Operations, iniciar cambio a MainMenu o Store, y verificar limpieza, action maps, foco y objetos persistentes. Cargar StoreInitial directamente y mediante slot. No deben quedar canvases duplicados.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

208. Campaña QA: build externa

Ejecutar build Windows x64 limpia, verificar resolución/modo, input, Quit, archivos de preferencias, Player.log y rendimiento. Repetir Golden Path y siete días según gate. La conducta solo Editor se clasifica S1/S2 según impacto.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

209. Campaña QA: contenido dinámico

Probar listas vacías, una fila, muchas filas, nombres largos, valores grandes, importes negativos, estados desconocidos y updates rápidos. Confirmar scroll, foco, performance y no overflow.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

210. Campaña QA: acciones idempotentes

Doble Submit/click en New Game, Order, Receive, Checkout, Close Day, Delete y Return. Verificar una sola operación, button disabled/loading y resultado consistente. Registrar transaction/ID cuando aplique.

La campaña produce ejecución, evidencia, defectos y recomendación de gate. Un hallazgo no se cierra sin reejecución atribuible a una versión.

211. Severidad específica de defectos UI

La severidad se deriva del impacto, no de lo visible que sea el defecto.

Severidad	Ejemplos UI	Gate
S0	Borrar/corromper save por acción UI, bloqueo total sin salida, seguridad	Bloquea todo
S1	Golden Path imposible, build sin input, modal irrecuperable, StoreInitial sin UI crítica	Bloquea Sprint/H6

Severidad	Ejemplos UI	Gate
S2	Click-through, foco roto en flujo importante, 720p bloqueado, error confuso con workaround	Debe corregirse o aceptarse formalmente
S3	Desalineación, copy menor, hover inconsistente, clipping no crítico	Planificable
S4	Mejora estética, microespaciado, polish no funcional	Backlog/visión

212. Checklist de aprobación de pantalla

La pantalla está vinculada a requisitos; muestra estado real; cubre nominal, vacío, loading, blocked y error; funciona con ratón y teclado; conserva foco; Cancel actúa sobre la capa superior; no propaga input; escala 80–150; pasa 720p/1080p; admite expansión; no depende solo del color/audio; usa tokens; strings están localizados o exceptuados; assets tienen licencia; no genera warnings; y cuenta con capturas y ejecución.

Una pantalla con una excepción documentada puede aprobarse **PASS WITH OBSERVATION** solo si no amenaza Golden Path, datos, accesibilidad mínima ni distribución. La excepción incluye owner y fecha/gate de resolución.

213. Checklist de candidata H6 para UI

Antes de H6: integrar StoreInitial; eliminar o exceptuar UI técnica; cerrar click-through; ejecutar suite completa; completar matriz de resoluciones/escalas; revisar ES/EN o alcance aprobado; verificar accesibilidad; probar audio silenciado; completar Golden Path y campaña de siete días; revisar Player.log; archivar build/checksum; actualizar QA, producción y trazabilidad; y emitir decisión formal.

No se aprueba H6 porque el HUD “se ve bien” en una captura. La candidata debe demostrar continuidad, control, recuperación, input exclusivo y legibilidad en build.

214. Matriz de prioridad, convivencia y descarte de mensajes

La interfaz no debe tratar todos los mensajes como equivalentes. Cartridge & Cloud combina confirmaciones locales, cambios económicos, avisos de flujo diario, errores persistentes, tutorial y estados de sistema; si todos ocupan el mismo canal, el jugador pierde información o recibe una secuencia ruidosa de toasts.

Prioridad	Familia	Ejemplos	Superficie preferente	Persistencia	Regla de interrupción
P0 crítica	integridad y bloqueo	save corrupto, autosave fallido sin recuperación, estado imposible	modal o banner persistente	hasta acción o resolución	interrumpe feedback no crítico
P1 operativa	acción requerida	checkout bloqueado, falta de stock, cierre no permitido	panel contextual + mensaje	hasta que cambie la causa	puede sustituir mensajes P2/P3
P2 confirmación	acción completada	pedido recibido, producto asignado, venta confirmada	toast o cambio de estado local	breve	se agrupa si se repite
P3 informativa	progreso y orientación	tutorial, ayuda, recordatorio	panel, tooltip o guía	contextual	nunca tapa P0/P1
P4 ambiental	sabor y detalle	comentario de cliente, microfeedback	mundo o elemento secundario	efímera	se descarta bajo carga

La cola de mensajes debe aplicar deduplicación por familia y entidad. Diez avisos iguales de reposición no deben generar diez elementos visuales; deben consolidarse en una única notificación con cantidad o resumen. Los errores persistentes no deben repetirse cada frame. Un cambio de panel debe limpiar tooltips y feedback cuyo contexto dejó de existir, pero no debe ocultar un fallo crítico que todavía exige respuesta.

La duración visual debe depender de legibilidad y acción, no solo de una constante global. Los textos breves de confirmación pueden desaparecer antes; un mensaje que explica cómo reparar un error debe permanecer o ser recuperable desde historial. `UiAccessibilitySettings.MessageDurationSeconds` ofrece una base útil, pero el sistema final debe separar duración base, prioridad y requisito de confirmación.

Cada mensaje requiere: ID estable; prioridad; origen; entidad afectada; texto localizable; alternativa no cromática; sonido opcional; política de deduplicación; duración; condición de cierre; y evidencia de que no provoca click-through ni roba foco indebidamente.

215. Versionado de tokens, componentes y contratos visuales

Los tokens de color, tipografía, espaciado, radios, bordes, tamaños mínimos y duraciones son contratos de diseño. No deben cambiarse mediante sustituciones aisladas en cada pantalla. Un cambio de verde corporativo, tamaño base o padding debe quedar registrado como una revisión del sistema, con impacto evaluado sobre HUD, Operations, modales, menús, estados disabled y capturas de referencia.

Se propone una convención de versión semántica para el sistema de UI:

- **UI 1.x:** sistema representativo del vertical slice, uGUI y pantallas runtime.
- **Cambio patch:** corrección visual o de copy sin alterar jerarquía ni interacción.
- **Cambio minor:** nuevo componente, estado o token compatible.
- **Cambio major:** migración de framework, sustitución de tipografía base, nueva arquitectura de navegación o ruptura de contratos de pantalla.

Cada token debe tener nombre semántico, no un nombre basado únicamente en el valor: `Color.Surface.Primary`, `Color.Action.Positive`, `Spacing.Panel`, `Typography.Body`, `Motion.ToastEnter`. Esto permite cambiar valores sin reescribir la intención. Los componentes deben consumir tokens y no duplicar literales. Durante la transición, los valores serializados o codificados deben inventariarse para evitar que convivan dos verdes, dos alturas de botón o dos jerarquías tipográficas sin justificación.

La revisión de un token debe incluir capturas comparativas, pruebas de contraste, escalado 80–150 %, resolución mínima admitida, strings largas y estado disabled. La trazabilidad debe enlazar el cambio con `19_UI_Style_Guide.md`, el código o prefab afectado, casos QA y build. No se considera cerrado por compilar: debe demostrar que la modificación no altera foco, hitboxes, legibilidad ni prioridad funcional.

Los tokens históricos de v0.3 se preservan como genealogía. Los valores implementados en `Sprint15UiFactory` y `Phase1RuntimeUiFactory` son la realidad actual. Cuando difieren, la guía marca objetivo, implementación y deuda, evitando llamar “estándar vigente” a un color que solo existe en un mockup antiguo.

216. Estrategia de migración desde UI generada en runtime

La UI actual fue creada en código para cerrar rápidamente la vertical slice y demostrar flujo funcional. Esa decisión es válida como baseline técnica, pero no obliga a mantener indefinidamente una construcción monolítica y difícil de previsualizar. La migración debe ser incremental y no introducir un segundo sistema de navegación sin control.

Orden recomendado:

1. inventariar todos los controles generados por `Sprint15UiFactory` y `Phase1RuntimeUiFactory`;
2. identificar componentes repetidos: botón, panel, encabezado, fila de datos, toast, modal y tab;
3. extraer tokens compartidos sin cambiar comportamiento;
4. crear prefabs uGUI representativos para componentes de mayor repetición;
5. sustituir una pantalla piloto manteniendo el mismo contrato de entrada/salida;
6. ejecutar pruebas de foco, input exclusivo, localización y escalado;
7. retirar el generador equivalente solo después de demostrar paridad;
8. documentar la sustitución y conservar rollback.

TextMeshPro es una migración razonable para tipografía de producción, pero debe introducirse con fuente y licencia registradas, fallback, atlas, cobertura de caracteres ES/EN y pruebas de wrapping. No debe aplicarse como reemplazo masivo justo antes de H6 sin campaña de regresión. `LegacyRuntime.ttf` puede seguir funcionando como fallback técnico mientras la tipografía final no esté aprobada.

La migración no debe mezclar en una misma tarea: cambio de framework, rediseño visual, nueva navegación, localización y lógica de negocio. Cada dimensión debe poder aislarse y revertirse. Un prefab visual no debe ejecutar transacciones de dominio; recibe view models, emite intenciones y presenta resultados.

Mientras convivan UI runtime y prefabs, debe existir una única autoridad de navegación y una sola puerta de input. Los canvases técnicos de escena, `Phase10OperationsScreen`, `MainMenuSlotScreen` y `StoreHudScreen` requieren sorting orders documentados y ausencia de solapamientos invisibles. Cualquier canvas legado que no sea parte de la experiencia candidata debe retirarse, deshabilitarse o quedar exceptuado con evidencia.

217. Criterios para elegir uGUI, UI Toolkit o una solución híbrida

La existencia de UI Toolkit en Unity no implica que el proyecto deba migrar automáticamente. La decisión se toma por riesgo, capacidad de previsualización, accesibilidad, navegación, rendimiento y coste de estabilización.

uGUI es la opción vigente para el vertical slice porque ya está integrado, probado y conectado con `InputSystemUIInputModule`. Es adecuado para HUD, modales, menús y paneles actuales. Sus principales deudas son la construcción runtime, estilos duplicados y ausencia de una librería de prefabs.

UI Toolkit puede evaluarse después de H6 para herramientas densas o pantallas de gestión futuras, siempre que se demuestre navegación, escalado, localización, gamepad, integración con Input System y paridad visual. No debe introducirse durante Sprint 16 o Sprint 17 por moda tecnológica.

Solución híbrida solo se acepta con fronteras claras: por ejemplo, uGUI para HUD y mundo, UI Toolkit para una herramienta aislada posterior. No se permite que dos sistemas compitan por focus, Escape/Cancel o captura de puntero. La arquitectura debe definir quién posee la capa activa, cómo se bloquea el mundo y cómo se restaura el contexto.

La decisión requiere ADR cuando cambia framework o navegación global. La prueba piloto debe incluir: resolución base y mínima; escalado 80/100/150 %; teclado y ratón; mando si entra en alcance; localización ES/EN; perfilado; captura de memoria; reapertura tras save/load; y recuperación de foco. El criterio no es “se ve más moderno”, sino menor riesgo total y mejor mantenibilidad sin romper la experiencia aprobada.

218. Conservación de las pantallas históricas de visión futura

La UI Style Guide v0.3 describía interfaces para ecommerce, publishing, desarrollo interno, plataforma, infraestructura, mercado y analítica. Esas pantallas son parte de la visión histórica y aportan lenguaje de producto, pero no forman parte del compromiso del vertical slice ni autorizan su implementación inmediata.

La consolidación aplica tres reglas:

1. preservar la intención y los patrones reutilizables;
2. marcar explícitamente el estado `VISION / NOT OPEN`;
3. impedir que mockups o tablas históricas se conviertan en backlog activo sin una decisión de alcance.

De ecommerce pueden reutilizarse patrones de catálogo, filtros y estados de pedido. De publishing, tablas de proyectos, hitos y contratos. De desarrollo interno, progreso, recursos y riesgo. De plataforma e infraestructura, estados de servicio, capacidad y alertas. Sin embargo, cada sistema futuro deberá redactar sus propios requisitos, modelo de datos, UX Flow, riesgos, QA y economía antes de reutilizar una pantalla histórica.

No se deben incluir botones deshabilitados que anuncien sistemas futuros en la candidata H6 salvo que el GDD y UX lo autoricen; crean expectativas y ruido. Tampoco se deben conservar tabs vacíos “para más adelante” en Operations. Las capacidades no abiertas permanecen documentadas, no expuestas al jugador.

Cuando una fase futura se autorice, la guía histórica sirve como semilla, no como especificación suficiente. Debe revisarse contra identidad actual, accesibilidad, localización, input y framework vigente. Esta disciplina protege el alcance sin perder el trabajo conceptual anterior.

219. Plan de madurez de UI por fase

Fase	Objetivo	Resultado mínimo	Exclusiones
Estado actual	flujo funcional demostrable	menú, slots, HUD, Operations, modales, audio básico y accesibilidad parcial	presentación final, localización completa, gamepad certificado
Sprint 16	integración representativa	UI legible sobre <code>StoreInitial</code> , sin placeholders visibles no exceptuados y con coherencia arte/audio	expansión de sistemas
Sprint 17	estabilización	click-through cerrado, foco robusto, regresiones, escalado, copy, rendimiento y campaña externa	rediseño total de framework

Fase	Objetivo	Resultado mínimo	Exclusiones
H6	candidata auditable	Golden Path y siete días, build externa, Player.log, evidencia, S0/S1 cerrados según política	visión futura
Post-H6	producto ampliable	librería de componentes, localización completa, opciones avanzadas, evaluación de mando y nuevas pantallas	compromisos no aprobados

La madurez no se mide por número de paneles. Se mide por continuidad: el jugador entiende estado y siguiente acción; la UI no ejecuta acciones de mundo; un error ofrece recuperación; el foco vuelve al lugar correcto; el texto cabe; los sonidos tienen alternativa visual; y la build mantiene el comportamiento observado en Editor.

Toda solicitud que añada una pantalla durante Sprint 16/17 debe demostrar que corrige una necesidad del gate. Una pantalla nueva no es automáticamente progreso: puede aumentar superficie de fallo, copy, localización, navegación y QA. La decisión debe compararse con mejorar una pantalla existente o resolver la información mediante el mundo.

220. Paquete de evidencia de una candidata UI para H6

El paquete de evidencia debe permitir que otra persona reproduzca la aprobación sin depender de memoria oral. Debe contener, como mínimo:

- build ID, versión, commit/SHA y checksum;
- resolución, modo de ventana y escala de UI utilizados;
- capturas de MainMenu, slots, HUD, cada tab relevante, pausa, confirmaciones, error y cierre diario;
- vídeo o secuencia que demuestre click de UI sin movimiento de mundo;
- prueba de Escape/Cancel y retorno de foco;
- matriz de 80 %, 100 % y 150 % de escala;
- prueba con textos largos ES/EN o justificación de alcance;
- audio UI activado, silenciado y con canales separados;
- lista de canvases activos y sorting orders;
- resultados EditMode y PlayMode;
- Golden Path y campaña de siete días;
- Player.log revisado;
- defectos conocidos y decisión formal.

Las capturas deben proceder de la build candidata, no de mockups. Un error mostrado debe incluir el estado previo, la acción que lo provoca y la recuperación. La evidencia de accesibilidad debe demostrar operación, no solo existencia de un toggle. La prueba de escalado debe incluir modales y tablas, no únicamente HUD.

El paquete debe enlazarse desde QA, producción y trazabilidad. Si una evidencia se sustituye, la anterior se conserva con estado histórico. No se aprueba una candidata con archivos sueltos imposibles de asociar a una versión exacta.

221. Retrospectiva y mantenimiento del sistema de interfaz

Después de cada sprint o candidata relevante, la retrospectiva de UI debe revisar: defectos escapados; mensajes confusos; pasos repetidos; componentes duplicados; hardcoded strings; problemas de foco; incidencias de input; coste de actualizar estilos; y evidencia difícil de reproducir.

Cada observación se clasifica como defecto, deuda, mejora o aprendizaje. Las acciones deben ser pequeñas y verificables: extraer un token, añadir una prueba de navegación, renombrar un evento, documentar sorting order o retirar un canvas. “Refactorizar toda la UI” no es una acción utilizable sin descomposición.

El mantenimiento periódico incluye:

- comparar guía, UX Flow, QA y código;
- revisar rutas y assets huérfanos;
- comprobar que no reaparecen strings directas;
- verificar licencias de fuentes e iconos;
- ejecutar resolución mínima y escala máxima;
- revisar localización y wrapping;
- comprobar logs por referencias ausentes;
- actualizar catálogo de deuda;
- archivar capturas de la versión vigente;
- registrar cambios en `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

La guía se revisará cuando cambie una regla, no por cada ajuste menor de contenido. El código y los assets son evidencia de implementación; el documento conserva intención, contratos y procedimiento. Cuando difieran, se abre una discrepancia y se decide qué fuente debe corregirse. Esta práctica evita que la guía se convierta en una descripción ideal desconectada del juego o, en el extremo contrario, en una simple transcripción del código actual.

Fin del documento.

Fuente: 19_UI_Style_Guide.md · SHA-256: `b34ca4db1eba7a536466b6a9a105d8d92396ade2108aa7dd99fa7d5d4eaebc1a`

Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.